大模型训练与推理工程详解

原理 · 流程 · 工程——写给转型 AI Infra 的分布式工程师

《大模型与 RAG 基础数学知识手册》《注意力模型与主流大模型详解》姊妹篇

面向分布式后台工程师的 AI Infra 转型读本

2026 年 10 月

目录

第 1 章 从分布式后台到 AI Infra：换个战场，不换基本功 3

第 2 章 训练的原理：一次参数更新的完整解剖 9

第 3 章 训练全流程：从原始网页到可用模型 16

第 4 章 分布式并行：把 70 亿参数放进集群 24

第 5 章 训练工程：混合精度、稳定性与省钱 32

第 6 章 推理原理：Prefill、Decode 与 KV Cache 42

第 7 章 推理工程：从 vLLM 到 PD 分离 50

第 8 章 集群、成本与转型路线图 59

第 1 章 从分布式后台到 AI Infra：换个战场，不换基本功

如果你在后台做了八年分布式系统，大概率已经对这些问题形成了肌肉记忆：缓存击穿怎么办、分片键怎么选、消息积压怎么消、P99 延迟怎么压、容量规划怎么做。现在你要转型 AI Infra，第一件事不是去背反向传播的公式，而是先回答一个问题：**这个新世界和我熟悉的旧世界，到底哪里一样、哪里不一样？**

这一章就回答这个问题。我们会做四件事：先对比两个世界的根本差异（1.1），再认识大模型系统的三个主角——数据、算力、参数，并算清第一笔账：为什么 GPU 的显存带宽比 CPU 内存带宽贵出一个数量级（1.2）；然后把你最熟悉的 RPC、MQ、聚合操作逐个翻译成 AI Infra 的集合通信原语，并手算 ring AllReduce 的带宽公式（1.3）；最后给出全书地图，告诉你后面七章各自解决什么问题（1.4）。

读完这一章你会得到一个核心结论：**AI Infra 并没有发明新的分布式难题，它只是把旧难题换了一套资源约束**。你的基本功——分片、副本、聚合、容错、容量规划、SLO——全部有效，只是作用的坐标轴从「磁盘与网卡」换成了「显存与 NVLink」。

1.1 两个世界：IO 密集 vs 计算密集

先从三个最刺眼的差异说起。

差异一：IO 密集 vs 计算密集。 典型的后台服务，CPU 大部分时间在等待：等数据库返回、等下游 RPC、等磁盘刷盘。 profiling 一台在线服务，CPU 利用率能到 40% 就算繁忙。瓶颈在 IO，优化手段是缓存、连接池、异步化。而大模型训练是纯粹的计算密集：GPU 在一个 iteration 里持续做矩阵乘法，理论峰值是单卡约 10^{15} FLOPS，工程师的核心 KPI 是 MFU（算力利用率）——让 GPU 真正用于计算的时间占比尽量逼近 100%。一个怕「等」，一个怕「停」。

差异二：无状态 vs 万亿参数有状态。 后台架构的第一课就是「服务无状态化」：状态外置到 Redis 和 DB，机器挂了随便换，扩缩容就是加减容器。而训练任务是人类工程史上最「有状态」的分布式程序之一：一个 7B 模型，仅优化器视角的状态就是 fp32 主权重、fp32 梯度、一阶动量、二阶动量各 4 字节，合计每参数 16 字节，即 $7 \times 10^9 \times 16\ \text{B} = 112\ \text{GB}$（附录 A2），这还没算激活值。这 112 GB 状态被切散在几十上百张显卡的显存里，任何一张卡宕机，整个任务的状态都不完整。「换个容器接着跑」在这里不成立，取而代之的是 checkpoint 机制——这是第 5 章的主题。

差异三：毫秒级 vs 数月长跑。 在线服务的 SLO 以毫秒计：P99 延迟 200 ms，超时重试 3 次，用户无感知。训练任务以周和月计：按附录 A5 的账，7B 模型、2T tokens 的总算力需求是 8.4×10^{22} FLOPs，1024 张卡也要跑约 1.9 天；真实的前沿模型是数万卡跑数月。在这么长的时间尺度上，故障不是小概率事件而是必然事件——1024 卡的集群，单卡年故障率即使只有几个百分点，摊下来也是平均几天就坏一张卡。后台的容错语义是「重试」，训练的容错语义是「回滚到上一个 checkpoint 接着跑」，两者的成本模型完全不同。

把更多维度摆在一起，就是这张对照表——建议把它贴在工位上，它是全书反复使用的「翻译词典」：

维度	分布式后台	AI Infra（大模型训练/推理）
瓶颈资源	磁盘 IO、网络 IO、DB 连接	GPU 算力、显存容量、显存/互联带宽
服务状态	无状态，状态外置	强有状态（参数 + 优化器 + KV cache）
工作单元	一次 RPC/一条消息（毫秒级）	一个 iteration（秒级）、一个训练任务（周月级）
扩容方式	加实例，负载均衡自动分流	改并行拓扑（第 4 章），不是加机器那么简单
失败语义	重试、熔断、降级	checkpoint 回滚、弹性训练（第 5 章）
性能指标	QPS、P99 延迟、错误率	MFU、吞吐（tokens/s）、TTFT/TPOT（第 6 章）
一致性	副本一致性、分布式事务	梯度同步的数值一致性（AllReduce，本章 1.3）
容量规划	峰值 QPS × 单机容量	显存账本 + 算力账本（A2/A5，第 4、8 章）

注意最后一行。后台的容量规划你闭着眼睛都会做：峰值 QPS 除以单机容量，再加冗余。AI Infra 的容量规划本质上是同一门手艺，只是要算两本新账——**显存账本**（这个模型的状态一共多少字节，要几张卡才放得下）和**算力账本**（总共多少 FLOPs，多少卡要跑多久）。本书附录 A 的所有数字，就是这两本账的范本，后面每一章都会反复引用。

1.2 大模型系统的三个主角：数据、算力、参数

任何一个大模型系统，拆开看都只有三个主角在相互作用：**数据**被喂进来，**算力**驱动计算，**参数**被反复读写和更新。搞清楚这三者的量级，后面所有工程决策都是顺水推舟。

数据：不是「数据库里的行」，而是「流动的语料」

后台工程师习惯的数据是结构化的：一张订单表几千万行，热点数据进缓存。训练数据则是数万亿 token 的非结构化语料（本书算例用 2T tokens），存放在对象存储里，经清洗、去重、打包后由高吞吐管道持续喂给 GPU（第 3 章展开）。这里的工程问题你很熟悉——ETL、去重、分片、流水线——只是规模换了一个数量级：GPU 吃东西的速度极快，数据管道一旦供不上，每秒烧的都是真金白银的卡时。这和你当年优化「DB 慢查询拖垮服务」是同构的问题：**让最贵的资源永远不要因为等数据而空转**。

参数：一个 7B 模型到底多大

「7B」指模型有 7×10^9 个参数，每个参数就是一个浮点数。推理时用 bf16 精度存储，每个参数 2 字节：

$$\text{权重体积} = 7 \times 10^9 \times 2\ \text{B} = 14\ \text{GB}$$

14 GB，单张 80 GB 显存的卡轻松放下——所以推理 7B 模型可以单卡搞定。但训练时按附录 A2 的口径要存约 112 GB（fp32 主权重 + 梯度 + 两个动量），一张卡就装不下了。这个「14 GB 能放下、112 GB 放不下」的落差，是第 4 章所有并行技术存在的根本原因。参数之于大模型系统，相当于数据库之于后台系统：**它既是计算的对象，也是一切容量问题的源头**。

算力：为什么主角是 GPU 而不是 CPU

按本书的五步结构来分析这个新概念。

直觉。 CPU 是几个到几十个强大的核，擅长复杂的串行逻辑；GPU 是成千上万个简单核，采用 SIMT（单指令多线程）执行模式——同一时刻所有核执行同一条指令、操作不同数据。神经网络的计算 95% 以上是矩阵乘法，天然是「同一个操作施加在海量数据上」，和 SIMT 严丝合缝。

类比。 这就像后台系统里的「批处理 vs 逐条处理」：CPU 是一个经验丰富的客服，一次处理一个复杂工单；GPU 是一万个实习生，每人发一张同样的表格同时填写。工单逻辑复杂时客服赢，表格量大且格式统一时实习生军团碾压。

数学与手算。 真正锁死 GPU 地位的，除了核数，还有一个常被忽视的硬件参数：**显存带宽**。大模型推理的 decode 阶段，每生成一个 token 都要把全部权重从显存读一遍。H100 级 GPU 的 HBM 显存带宽约 3 TB/s，而 CPU 的 DDR 内存带宽约 0.3 TB/s（300 GB/s）。读一遍 14 GB 的 7B 权重，两者分别需要：

$$t_{\text{GPU}} = \frac{14\ \text{GB}}{3000\ \text{GB/s}} \approx 0.0047\ \text{s} \approx 4.7\ \text{ms}$$

$$t_{\text{CPU}} = \frac{14\ \text{GB}}{300\ \text{GB/s}} \approx 0.047\ \text{s} \approx 47\ \text{ms}$$

反过来换算成吞吐上限：单卡单请求下，生成速度的理论天花板就是「带宽 ÷ 模型大小」——GPU 约 $3000/14 \approx 214$ tokens/s，CPU 约 $300/14 \approx 21$ tokens/s。**带宽差 10 倍，token 生成速度的上限就差 10 倍**，而实际差距因为核数差异只会更大。这还没提 HBM 与显存颗粒紧贴着计算单元的封装成本——GPU 贵，一半贵在算力，另一半就贵在这条带宽上。

工程含义。 记住这个视角：**大模型系统的很多性能问题，本质是「搬数据」而不是「算数据」**。Decode 阶段卡在显存带宽（第 6 章），集合通信卡在互联带宽（本章 1.3），FlashAttention 优化的是 SRAM 与 HBM 之间的搬运次数（第 7 章）。你在后台时代养成的「先找瓶颈在哪一层 IO」的直觉，在这里依然是第一分析框架——只是 IO 的层次从 内存 → 磁盘 → 网络 变成了 寄存器 → SRAM → HBM → NVLink → IB 网络。

1.3 通信原语迁移：RPC → 集合通信

现在来到后台工程师最亲切的一节：通信。

从一对一、一对多到「全员协同」

后台系统的通信模式你烂熟于心：RPC 是一对一请求响应，MQ 是解耦的生产消费，配置中心推送是广播，MapReduce 的 shuffle 是聚合。AI Infra 不用 RPC 也不用 MQ，它用一套叫**集合通信**（collective communication）的原语，由 NCCL 等库实现，直接操作 GPU 显存、走 NVLink 或 IB verbs，不经过 TCP 协议栈、不序列化对象——因为训练时每秒钟要搬 GB 级的梯度，任何一次多余的内存拷贝都是犯罪。

好消息是，每个集合通信原语都能在你的旧知识体系里找到对应物：

集合通信原语	语义	分布式后台对应物	典型用途
Broadcast	一台机器的数据发给所有人	配置中心推送、缓存失效广播	分发模型参数、输入数据
Reduce	所有人的数据聚合成一份，结果留在主节点	MapReduce 的 reduce 阶段	梯度求和（只留结果时）
AllReduce	所有人的数据聚合，结果人人有份	reduce + 结果广播回所有节点	数据并行的梯度同步（第 4 章）
AllGather	每人拿出自己的一片，拼出完整数据，人人有份	从所有分片拉数据做全量视图	ZeRO/FSDP 取回完整参数（第 4 章）
ReduceScatter	先聚合再切分，每人拿走结果的一片	聚合结果按分片键路由回各分片	ZeRO/FSDP 回传梯度分片（第 4 章）

表中真正的主角是 AllReduce。数据并行训练时，每张卡用不同的数据切片各自算出梯度，然后必须把所有卡的梯度求平均、且每张卡都拿到平均结果，才能继续下一步——这正是 AllReduce 的语义。它之于训练，相当于 MQ 之于电商系统：不理解它，就看不懂任何一张训练架构图。

Ring AllReduce：直觉、类比、公式、手算

直觉。 P 张卡，每张卡上有一个长度为 M 字节的梯度向量，目标是每张卡都拿到「所有卡梯度的和」。最朴素的办法是找一个中心节点：所有人把 M 字节发给它，它求和后再广播回去。这和早期大数据系统的参数服务器架构一模一样，问题也一样——中心节点要收 $P \times M$ 字节，成为单点瓶颈。

类比。 Ring AllReduce 把 P 张卡首尾相连围成一个环，通信分两个阶段：第一阶段（ReduceScatter）每轮每卡只把向量的 $1/P$ 发给右邻居、从左邻居收 $1/P$，转 $P-1$ 圈后，每卡手里恰好握有「向量的 $1/P$ 的全员求和结果」；第二阶段（AllGather）再转 $P-1$ 圈，把各自手里的分片沿环传递补齐，最终人人有完整结果。这就像开会时不用每个人上台向全场发言，而是围坐一圈传纸条：纸条每传一轮汇总一次，$2(P-1)$ 轮后全场的信息人人有份，而每个人经手的纸条数量与会议室人数几乎无关。

数学。 两个阶段各传 $P-1$ 轮、每轮每卡发送（同时接收）M/P 字节，所以每卡的发送量与接收量都是：

$$\text{每卡发送量} = \text{每卡接收量} = 2 \times \frac{P-1}{P} \times M$$

注意这个公式的美妙之处：当 P 变大时，$\frac{P-1}{P}$ 趋近于 1，每卡通信量趋近于 $2M$，**与集群规模基本无关**。而朴素参数服务器方案里中心节点要收 $P \times M$。这就是为什么千卡集群的梯度同步都用 ring（或其树形变体），而不是中心聚合——和你当年「去掉单点瓶颈、把负载摊到环上」的架构直觉完全一致。

手算例子（附录 A2/A3 数字）。 一台 8 卡节点，$P=8$；7B 模型 bf16 梯度，$M=14$ GB。代入公式：

$$\text{每卡发送量} = 2 \times \frac{8-1}{8} \times 14\ \text{GB} = 2 \times \frac{7}{8} \times 14\ \text{GB} = 24.5\ \text{GB}$$

这 24.5 GB 要走什么链路？分两种情形：

- 全部在**机内** NVLink（约 900 GB/s）上跑：$t = 24.5/900 \approx 0.027\ \text{s} \approx 27\ \text{ms}$；
- 全部走**跨机** IB 网络（400 Gb/s，即 50 GB/s）：$t = 24.5/50 = 0.49\ \text{s} \approx 490\ \text{ms}$。

同样是 24.5 GB，机内 27 ms、跨机 490 ms，**差约 18 倍**——正好是 900 与 50 的带宽之比。作为对照，7B 模型一个 iteration 的计算量约 $6ND$ 中「每 token $6N$ FLOPs」的尺度，单卡算力 10^{15} FLOPS 下计算本身也是几十到几百毫秒量级：也就是说，通信如果走机内，可以被计算掩盖或只占小头；一旦被迫走跨机慢链路，通信就直接成为主角，GPU 大量时间在等梯度。

工程含义。 这就是为什么**通信拓扑决定并行策略**：通信频繁且量大的张量并行（TP，每层都要 AllReduce）必须局限在机内 NVLink 域；通信相对稀疏的数据并行（DP，每个 iteration 才 AllReduce 一次）可以跨机。第 4 章的「TP 机内、PP 跨节点、DP 最外层」这条经验法则，到这里已经能自己推导出来了——它不是背诵的口诀，而是 900 GB/s 与 50 GB/s 这两个数字的必然推论。

1.4 全书地图：问题清单与章节索引

最后，把整本书要讲的问题摆成两张清单。你以后在生产环境里遇到的几乎每一个 AI Infra 问题，都能在这里找到格子。

训练侧问题清单（第 2–5 章）：

问题	一句话描述	你的旧知识对应物	章节
原理	一次参数更新到底发生了什么	一次「写请求」的完整路径	第 2 章
数据	2T tokens 怎么清洗、去重、喂给 GPU	ETL 管道、消息积压治理	第 3 章

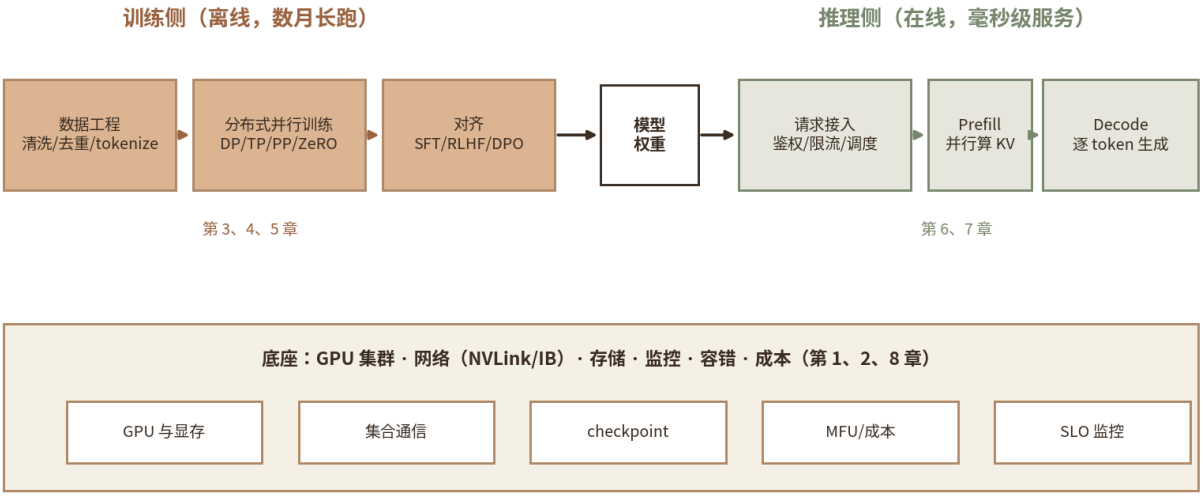
并行	112 GB 状态切散到上千张卡	分片 + 副本 + 一致性协议	第 4 章
显存	每字节省着花：混合精度、重算、分片	内存治理、缓存淘汰	第 4、5 章
稳定性	loss spike 的处置 playbook	线上故障应急手册	第 5 章
容错	千卡集群平均几天坏一卡，怎么不丢进度	故障恢复、数据备份	第 5 章
成本	MFU 50% 意味着一半的钱在空转	资源利用率优化	第 5、8 章

推理侧问题清单（第 6–8 章）：

问题	一句话描述	你的旧知识对应物	章节
延迟	TTFT 与 TPOT，首 token 快、后续 token 稳	首字节延迟 + P99 长尾	第 6 章
吞吐	continuous batching 让 GPU 不空转	连接复用、批量处理	第 7 章
显存	KV cache 分页与量化	操作系统的虚拟内存、数据压缩	第 7 章
调度	Prefill/Decode 分离，读写分离	数据库读写分离、冷热分离	第 7 章
集群	网络拓扑、存储、成本核算	IDC 容量规划、成本分摊	第 8 章
转型	技能迁移表与 90 天路线图	你自己的学习路径	第 8 章

两张清单合起来，就是本册的叙事主线：**训练侧追求「算力不空转」，推理侧追求「延迟可预期」，两者共享同一套显存与带宽的物理约束**。

回到本章开头那句话：换个战场，不换基本功。第 2 章我们会用一个 1 个参数的玩具神经元，把前向、反向、梯度下降的每一步数值都手算出来——那是理解后面所有工程技术的地基。但请带着本章的视角去读它：当你在第 2 章看到「梯度」时，心里想的应该是「这是一个 14 GB、要在 8 张卡之间 AllReduce 的消息」；当你看到「学习率调度」时，想的应该是「这是数月长跑任务的稳定性旋钮」。这样，ML 的数学对你来说就从来不是玄学，而是另一套你迟早会摸透的系统行为。



第 2 章 训练的原理：一次参数更新的完整解剖

做分布式后台的同行都熟悉：新系统上线前，要把一条请求从接入层到存储层的完整路径走一遍，之后所有优化和排查都建立在这条「请求时间线」上。训练大模型，本质上就是不断重复同一个动作：**一次参数更新**（一个 iteration）。千亿次迭代听起来吓人，但每次迭代的内部结构完全一样——前向传播算出预测，损失函数量化误差，反向传播算出梯度，优化器沿梯度更新参数。本章就解剖这一个 step：先用一个只有 2 个参数的玩具神经元把每个数字手算出来；再放大到 70 亿参数，推出全书最重要的算力公式 $6ND$；最后回到工程视角，看这一步在真实框架里如何被执行，以及通信（第 1 章的 AllReduce）插在时间线的哪个位置。

2.1 前向传播：从一次乘法开始

直觉。 模型的「预测」就是把输入送进一堆函数，得到一个输出。参数就是函数里待定的系数。训练之前系数是错的（随机初始化），预测自然离谱；训练就是不断修正系数的过程。

类比（分布式系统）。 前向传播像一次只读请求的处理链路：请求（输入 x）依次穿过网关、服务层、存储层（一层层神经网络），每层只做计算、不改任何状态，最后返回响应（预测 $\hat{y}$）。响应离用户预期有多远，由下游的「评价系统」（损失函数）打分。

数学。 本书的统一玩具算例（附录 A1）是一个不能再小的神经元：

$$\hat{y} = wx + b$$

它只有 2 个参数：权重 w 和偏置 b。初始值 $w = 0.5$，$b = 0$；训练样本只有一个：输入 $x = 2$，目标 $y = 3$。这个神经元的任务是：根据 $x = 2$ 预测出尽量接近 3 的输出。

手算例子。 前向传播就是代入：

$$\hat{y} = w \cdot x + b = 0.5 \times 2 + 0 = 1$$

预测是 1，目标是 3，差得不少。注意这里发生的一切只是「一次乘法加一次加法」——请记住这个观察，2.5 节推导 $6ND$ 时它是关键。

工程含义。 真实大模型的前向在结构上与这个玩具一模一样，只是「一层」从一次乘法变成矩阵乘法：输入是几千维向量，参数是几千乘几千的矩阵，输出 = 矩阵 × 向量，再加激活函数（第二册已详述），堆几十层。向量化后一次前向处理一整个 batch 的 token，但本质没变：仍是固定参数的纯函数计算、不写状态。这也是前向天然适合 GPU 的原因——海量无依赖的乘加，正是 SIMT 架构的主场（第 1 章）。

2.2 损失函数：给「差多远」一个数

直觉。 光知道「预测 1、目标 3」不够，机器需要一个数来量化「差多远」，而且这个数要满足两条：预测越离谱，数越大；对参数可导，这样才能知道「往哪个方向改参数会让它变小」。这个数就是损失（loss）。

类比（分布式系统）。 损失函数之于训练，就像 SLI（服务水平指标）之于运维：它把「系统表现好不好」压缩成一个可监控、可告警的标量。训练的 loss 曲线就是你的核心监控大盘，第 5 章讲稳定性时我们还会反复盯着它。

数学。 本章用最容易手算的均方误差（MSE）：预测与目标之差的平方，

$$L = (\hat{y} - y)^2$$

平方带来两个好性质：一是非负，预测偏高偏低都受罚；二是处处可导，$\hat{y}$ 偏离 y 越远，导数的绝对值越大——「错得越多，改正的力度越大」，这正是我们想要的。

手算例子。 代入 A1 的数字：

$$L = (\hat{y} - y)^2 = (1 - 3)^2 = (-2)^2 = 4$$

当前损失是 4。这个数字本身没有物理单位，它的意义在于**比较**：同一个模型，loss 从 4 降到 0.16 就是变好了。

工程含义。 大语言模型实际用的损失不是 MSE，而是交叉熵（cross-entropy）：损失是「正确下一个词的概率」的负对数。第一册已推导过交叉熵与 softmax 的细节，这里只说一句为什么大模型用它：预测下一个词本质是几万分类问题，交叉熵在「正确词的概率被压得很低」时给出极大梯度，纠错信号远比 MSE 强。但请记住：**损失函数可以换，『前向得到 $\hat{y}$ → 损失得到标量 L → 对参数求梯度』的骨架不变**。本章用 MSE 讲完骨架，结论全部平移。

2.3 反向传播：链式法则的一次完整手算

直觉。 现在我们手里有一个数 $L = 4$，以及两个参数 w、b。问题是：L 变大变小，分别有多少「责任」要算在 w 头上、多少算在 b 头上？把责任量化成数字，就是求偏导数 $\frac{\partial L}{\partial w}$ 和 $\frac{\partial L}{\partial b}$。反向传播就是高效算出所有参数偏导数的算法。

类比（分布式系统）。 这像极了故障定界：请求的最终延迟超标（L 很大），你要沿调用链反向回溯，算出每层各贡献了多少延迟。链上每一环的「贡献度」，就是反向传播里每层参数的梯度。而且定界时你不会对每一环都重发一次完整请求——反向传播的高明之处也在于**复用前向已算过的中间结果**，一次回溯拿到全部梯度。

数学。 只需要一条高中求导规则——链式法则：复合函数的导数等于各层导数相乘。我们把 L 到 w 的依赖链写清楚：

$$L = (\hat{y} - y)^2, \qquad \hat{y} = wx + b$$

L 依赖 $\hat{y}$，$\hat{y}$ 依赖 w 和 b。于是：

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}, \qquad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b}$$

三个「零件导数」分别是：

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y), \qquad \frac{\partial \hat{y}}{\partial w} = x, \qquad \frac{\partial \hat{y}}{\partial b} = 1$$

手算例子。 全程代入 A1 的数字（$w = 0.5$，$b = 0$，$x = 2$，$y = 3$，前向已算出 $\hat{y} = 1$）。这里把每个中间量都写出来——这正是反向传播的真实工作方式，框架里存下来的也是这些数：

第一步，损失对预测的导数（从输出端往回走的第一站）：

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = 2 \times (1 - 3) = 2 \times (-2) = -4$$

第二步，预测对 w 的导数：

$$\frac{\partial \hat{y}}{\partial w} = x = 2$$

第三步，两站相乘，得到 w 的梯度：

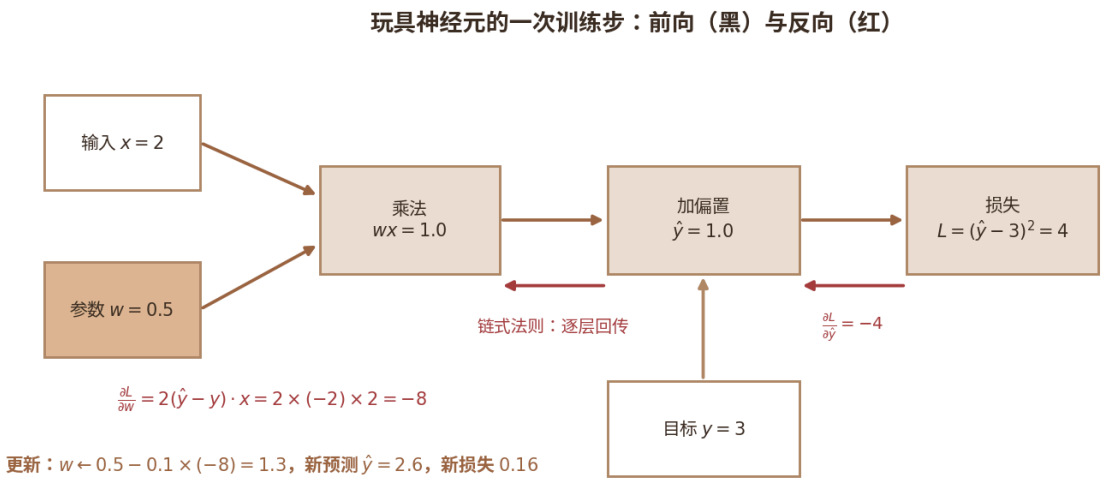
$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w} = (-4) \times 2 = -8$$

b 的梯度同样走这条路，只是第二站不同（$\hat{y}$ 对 b 的导数是 1）：

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b} = (-4) \times 1 = -4$$

两个梯度的含义：w 每增大 1 个单位，L 大约变化 -8 个单位，即 w 增大、损失减小；b 同理。注意梯度的**绝对值**还告诉你责任的轻重——w 的梯度绝对值是 b 的两倍，因为 w 的作用被 $x = 2$ 放大了。

再强调一次结构：中间量 $\frac{\partial L}{\partial \hat{y}} = -4$ 被 $\frac{\partial L}{\partial w}$ 和 $\frac{\partial L}{\partial b}$ **共用**了。玩具例子里只省一次乘法，但在 70 亿参数的模型里，这种「上游梯度被下游所有参数复用」的结构，就是反向传播只需约两倍前向代价就能算出全部梯度的原因（2.5 节细算账）。



2.4 梯度下降：迈出第一步

直觉。 梯度指出的方向是损失**上升**最快的方向；想让损失下降，就朝反方向走。这就是梯度下降的全部思想。走多大一步，由学习率（learning rate）η 控制。

类比（分布式系统）。 梯度下降像基于监控的容量调优：每次只朝「指标变好」的方向调一小步，观察一轮再调。步子太小（η 太小），几十轮才到位，浪费迭代；步子太大（η 太大），一下调过头，指标在目标两侧震荡甚至发散——和第 5 章的 loss spike 是同一个物理。

数学。 更新规则：

$$w' = w - \eta \cdot \frac{\partial L}{\partial w}$$

手算例子。 取 $\eta = 0.1$。本章采用「只更新 w、暂不更新 b」的版本（这样第二轮还留有小损失，便于观察训练过程；b 的更新完全同理）：

$$w' = 0.5 - 0.1 \times (-8) = 0.5 + 0.8 = 1.3$$

b 保持 0 不动。b 的梯度 -4 如果也参与更新，会得到 $b' = 0 - 0.1 \times (-4) = 0.4$，一步就把损失打到几乎为零，反而不方便演示——所以本章只用它说明「每个参数都有自己的梯度，各走各的更新」。

再前向验证。 用新参数重跑一次前向：

$$\hat{y}' = w' \cdot x + b = 1.3 \times 2 + 0 = 2.6$$

$$L' = (\hat{y}' - y)^2 = (2.6 - 3)^2 = (-0.4)^2 = 0.16$$

损失从 4 降到 0.16，一步降了 96%。预测从 1 涨到 2.6，离目标 3 更近。下一个 iteration 会基于 $w = 1.3$ 重新算前向、反向、再更新，如此往复，直到损失收敛到接近 0。

工程含义。 这里出现了两个超参数：学习率 η 和「一次用多少样本算梯度」（batch size，本章是 1 条）。真实训练里 η 不是常数，而是先 warmup 再 cosine 衰减的调度曲线（第 3 章）；优化器也不是裸梯度下降，而是带动量的 AdamW（第 4 章算它的显存账）。但骨架 $w' = w - \eta \cdot g$ 永远不变。

到这里，一个 iteration 的四个环节——前向、损失、反向、更新——已完整手算过一遍。下面放大到真实尺度。

2.5 从 1 个参数到 70 亿个：$6ND$ 的来历

现在回答全书最重要的容量规划问题：训练一个模型到底要多少计算？结论是经验公式：

$$\text{训练总 FLOPs} \approx 6ND$$

其中 N 是参数量，D 是训练 token 数，系数 6 是本节要推导的。FLOPs 指浮点运算次数（一次乘法或一次加法各算 1 次），FLOPS（大写 S）指每秒浮点运算次数，是算力单位。

第一步：矩阵乘的 FLOPs。 神经网络每层的核心是矩阵乘 $\hat{y} = Wx$。设矩阵 W 有 n 个元素（即 n 个参数），每算输出向量的一个分量，要把 W 某一行的每个元素各乘一次、再连加——即每个参数恰好参与**一次乘法和一次加法**，共 2 次浮点运算。经验法则：

$$\text{矩阵乘处理一个 token 的 FLOPs} \approx 2 \times \text{参数量}$$

玩具神经元正是如此：w 参与 1 次乘法、b 参与 1 次加法，2 个参数、2 次运算。激活函数、注意力等开销占比小，估算时忽略。

第二步：前向 $2N$。 前向传播每个 token 把每层矩阵乘跑一遍，合计约 $2N$ FLOPs/token。

第三步：反向 $4N$。 反向传播对每个参数要做**两次**矩阵乘量级的计算：一次算「梯度对输入的偏导」（把上游梯度穿过本层传下去，对应 2.3 节的 $\frac{\partial L}{\partial \hat{y}}$ 这类中间量），一次算「梯度对参数的偏导」（对应 $\frac{\partial L}{\partial w}$）。每次都是约 $2N$，合计约 $4N$ FLOPs/token。这就是为什么反向比前向贵一倍——它要同时给「下一层」和「参数」两个对象交账。

第四步：合计 $6N$。 前向 $2N$ + 反向 $4N$ = 每 token 约 $6N$ FLOPs，乘以训练集 token 数 D：

$$\text{总 FLOPs} \approx 6ND$$

手算例子（附录 A5）。 训一个 7B 模型（$N = 7 \times 10^9$），用 2T tokens（$D = 2 \times 10^{12}$）：

$$6ND = 6 \times 7 \times 10^9 \times 2 \times 10^{12} = 8.4 \times 10^{22} \text{ FLOPs}$$

换算成时间。单卡 H100 级 GPU 的 bf16 峰值约 10^{15} FLOPS，但实际达不到峰值——算力利用率 MFU 按 50% 计（第 5 章专讲 MFU），有效算力约 5×10^{14} FLOPS。单卡训练需要：

$$\frac{8.4 \times 10^{22}}{5 \times 10^{14}} = 1.68 \times 10^{8} \text{ 秒}$$

一年约 3.15×10^{7} 秒，所以：

$$\frac{1.68 \times 10^{8}}{3.15 \times 10^{7}} \approx 5.3 \text{ 年}$$

一张卡要训 5.3 年——这就是「为什么必须分布式」的定量回答。用 1024 张卡并行（假设线性加速，实际要打折）：

$$\frac{1.68 \times 10^{8}}{1024} \approx 1.64 \times 10^{5} \text{ 秒} \approx 1.9 \text{ 天}$$

工程含义。 $6ND$ 是训练侧的「容量规划第一公式」，地位堪比后台的「QPS × 平均处理时长 = 所需核数」。算力需求只与参数量和 token 数有关，与怎么切并行无关——并行只是把同一笔账分摊到更多卡上，把墙钟时间从 5.3 年压到 1.9 天。第 4 章的并行技术、第 5 章的省钱手段，最终都围绕这笔 8.4×10^{22} FLOPs 的账做文章。

2.6 工程视角：计算图、自动微分与一次 iteration 的时间线

手算是为了理解原理，真实训练里没有人手算梯度。这一节把 2.1–2.4 的数学翻译回工程系统。

计算图：把模型变成一张 DAG。 框架（PyTorch 等）执行前向时，把每个运算记成图的节点：玩具例子的图是 $w \xrightarrow{\times x} wx \xrightarrow{+b} \hat{y} \xrightarrow{-y} (\hat{y}-y) \xrightarrow{(\cdot)^2} L$。每个节点除记录运算类型外，还要**存下前向的中间值**（如 $\hat{y}=1$、$\hat{y}-y=-2$），因为反向要用。这立刻解释了一个工程师敏感的事实：**反向传播要求前向的激活值一直活在显存里**，层数越多、batch 越大，激活显存越惊人——第 4 章的显存账和第 5 章的梯度检查点都在对付这件事。

自动微分：反向传播是图的一次反向遍历。 调一行 `loss.backward()`，框架从 L 节点出发沿图反向遍历，每到一节就把「上游传来的梯度」乘以「本节点的局部导数」分发给上游——这正是 2.3 节手算的链式法则，由代码对每个算子机械执行。自动微分就是一个内置的、保证不出错的「责任分摊器」：工程师只定义前向，反向自动获得，且与手算严格一致。

一次 iteration 的时间线。 把本章所有环节按时间顺序排开，一个真实训练 step 长这样：

阶段	做什么	对应本章	资源特征
1. 数据加载	DataLoader 从存储读 batch、tokenize、搬到 GPU 显存	第 3 章详述	IO / 带宽密集，可与计算重叠
2. 前向传播	逐层算到 loss	2.1–2.2	计算密集（$2N$），产生激活值

3. 反向传播	自动微分算出全部梯度	2.3	计算密集（$4N$），消耗激活值
4. 梯度通信	多卡对梯度做 AllReduce	第 1 章原语，第 4 章详述	网络带宽密集
5. 参数更新	优化器按梯度更新 w	2.4	计算量小，读写优化器状态

通信插在反向和更新之间。 原因是：数据并行下每张卡只算了自己那份数据的梯度（第 4 章），必须先聚合平均，各卡才能拿到「全 batch 的梯度」去更新，否则参数会越走越散——这就是第 1 章 AllReduce 原语在训练里的落点：一次 AllReduce 把 N 个梯度值在所有卡间求和/平均。从时间线看，通信是反向结束、更新开始前的一段纯网络时间，不占算力却占墙钟，于是第 5 章的「通信计算重叠」（反向还在算后面几层时，先把已算出的梯度分桶异步 AllReduce）就成了关键优化。

类比收尾。 把一个 iteration 比作一次 MapReduce job：数据加载是输入分片，前向+反向是 map（各卡独立算局部梯度），AllReduce 是 reduce（全局聚合），参数更新是写回结果。你掌握的分布式直觉——分片、聚合、带宽瓶颈、计算通信重叠——在新战场全部有效，只是主角从「请求」换成了「参数与梯度」。

本章小结。 玩具神经元走完了一次参数更新的全部四步：前向 $\hat{y}=1$、损失 $L=4$、反向 $\frac{\partial L}{\partial w}=-8$（$b$ 的梯度 -4 同理）、更新 $w'=1.3$ 后损失降到 0.16。放大到 7B 参数、2T token，同一套机制对应 8.4×10^{22} FLOPs——单卡 5.3 年，1024 卡约 1.9 天。数学到此为止，剩下的全是工程：怎么把这 1.9 天真跑进集群，是第 3–5 章的主题。

第 3 章 训练全流程：从原始网页到可用模型

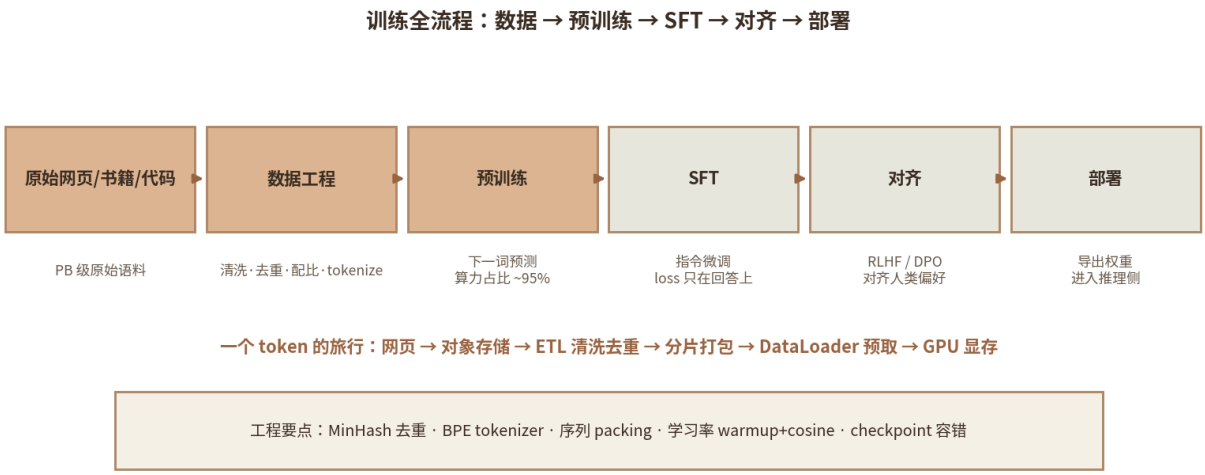
第 2 章我们解剖了一次参数更新：前向、损失、反向、更新，一个玩具神经元走完整个循环。但真实世界里，没有人用手推的方式训练 7B 模型。真实训练是一条长达数月的流水线：先把互联网上 PB 级的原始网页炼成几千亿 token 的干净语料，再让模型在 1024 张卡上读完这些语料（预训练），然后教它听懂指令（SFT），最后教它说人话、不说坏话（对齐）。本章把这条流水线从头到尾走一遍。对分布式后台工程师来说，好消息是：这条流水线的每一环都是你熟悉的东西——ETL、去重、分片、预取、调度——只是规模和数学目标换了一副面孔。

3.1 全景：四个阶段，一张账单

从原始网页到可用模型，标准路径是四个阶段：

阶段	输入数据	数据量级	训练目标	算力占比（经验值）
数据工程	原始网页/书籍/代码	PB 级原始文本	产出干净、去重、配比合理的 token 流	主要是 CPU/存储，GPU 占比约 0
预训练	数千亿到数万亿 token	2T tokens（本书统一算例）	下一词预测，学语言与世界知识	约 95%–99%
SFT（监督微调）	指令-回答对	数十万到数百万条	学会"听懂指令、按格式回答"	约 1%
对齐（RLHF/DPO）	人类偏好对比数据	数万到数十万条	让回答更有用、更无害	约 1%（RLHF 会更贵）

这张表回答了一个新人最常问的问题：为什么大家都说"预训练烧钱"？因为后面三个阶段加起来吃的算力，往往不到预训练的零头。回忆附录 A5 的账：7B 模型、2T tokens，训练 FLOPs 约 $6ND = 8.4 \times 10^{22}$，1024 张卡要跑约 1.9 天——这还只是 7B 和 2T 的"入门级"配置，主流模型的账单再乘以一到两个数量级。所以本章 3.2 和 3.3 是重点：数据质量决定模型上限，预训练决定账单大小；3.4 和 3.5 只讲工程骨架，对齐的数学推导已在第二册第 7 章完成，这里不重复。



3.2 数据工程：决定模型上限的不是算法，是数据

分布式后台里有一句老话：garbage in, garbage out。大模型把这句话放大到极致——模型只是"读完整个互联网并记住规律"的机器，你给它读什么，它就是什么。数据工程的目标很朴素：从 PB 级原始抓取里，炼出约 2T tokens 的干净语料。典型留存率低得惊人：从 Common Crawl 原始网页到最终进训练集的 token，常见只有 1% 到 5%。中间丢掉的东西去了哪里？我们按处理顺序看四个环节：清洗、去重、质量过滤、配比，外加两个把文本变成"模型能吃的数字"的关键组件：tokenizer 与序列 packing。

清洗：从 HTML 到纯文本

爬取的原始数据是 HTML，第一步是提取正文：去掉导航栏、广告、页脚、脚本。这一步和你写过的爬虫解析器没有本质区别，只是规模不同——用 Spark 或 Ray 集群跑，每天处理几十 TB。工程上要注意两点：一是正文提取规则直接决定语料质量，一套好的提取规则（保留代码缩进、公式、表格结构）比换个大模型架构更涨点；二是所有清洗规则必须可回放、可版本化，因为你总会在训练中途发现"某类脏数据没被洗掉"，需要重跑局部管道。

去重：MinHash 的直觉——签名撞车就是重复

互联网语料里重复内容多到离谱：转载、镜像站、模板页、SEO 农场，同一段文字可能出现几十次。不去重的后果不仅是浪费算力——模型会把重复内容"背下来"，加剧过拟合和数据泄漏（测试集的题出现在训练集里）。

精确去重（全文哈希）只能抓一字不差的复制，对付不了"改了几个字"的近似重复。近似去重的工业标准是 MinHash，属于局部敏感哈希（LSH）家族。它的思想对后台工程师来说非常亲切，就是 Bloom filter 的表亲：把一个巨大的对象压成一串很小的签名，用签名碰撞代替全文比对。

直觉三步走：

1. **切 shingles**：把文档切成连续的词片段（比如每 5 个词一片），一篇文档变成一个集合。
2. **压签名**：对集合做 k 次不同的哈希，每次取所有 shingle 哈希值的最小值，得到 k 个数——这就是这篇文档的签名。一篇 10 KB 的文档，被压成 128 个 64 位整数，只有 1 KB。
3. **比签名**：两篇文档的 Jaccard 相似度（集合交集的大小除以并集的大小）约等于"签名在同一个位置撞车"的概率。相似度高的文档，签名大量撞车。

手算一个例子感受放大效应。假设两篇文档真实相似度为 0.9（集合有 90% 重叠），那么签名单个位置撞车的概率就是 0.9。只看一个位置噪声太大，工程上把 128 个签名位置切成 32 组、每组 4 个位置（这叫 banding）。某组 4 个位置全部撞车的概率是：

$$P(\text{单组撞车}) = 0.9^4 = 0.6561$$

32 组里至少有一组撞车的概率是：

$$P(\text{被判为重复}) = 1 - (1 - 0.6561)^{32} \approx 1 - 0.3439^{32} \approx 1$$

也就是说，相似度 0.9 的文档几乎必然被抓出来；而相似度只有 0.5 的文档，单组撞车概率 $0.5^4 = 0.0625$，32 组全不撞的概率 $(1 - 0.0625)^{32} \approx 0.127$，仍有约 13% 漏网——所以阈值（多少相似度算重复）是个工程权衡：阈值低则误杀多（把正常引用也删了），阈值高则漏网多。和 Bloom filter 一样，这是拿可控的假阳性/假阴性换掉不可承受的存储与比对成本：全量两两比对是 $O(n^2)$，几亿篇文档根本跑不动；签名化之后，比对只在"同组撞车"的候选对之间发生，成本降到接近线性。

质量过滤与配比：给语料"挑食材、配菜谱"

去重之后还要质量过滤：用启发式规则（文本长度、标点比例、乱码率、敏感词）加一个小型分类模型，把"通顺、有信息量"的文本留下。这一步本质是训练一个二分类器当"质检员"，成本极低，收益极高——公开实验反复证明，过滤策略的差异足以让同等算力下的模型能力差出一截。

最后是**配比**（data mixture）：网页、书籍、代码、数学、多语言各占多少比例。这像消息队列里的加权轮询：每个数据源是一个队列，DataLoader 按权重采样。配比没有理论最优解，只有实验最优解——先小规模（比如 1B 模型配 20B tokens）扫几组配比，看下游 benchmark 再定大盘。这也是为什么头部团队把配比表当成核心机密。

Tokenizer：BPE 怎么把文本切成 token

模型不吃字符，吃 token ID。切 token 的主流算法是 BPE（Byte Pair Encoding，字节对编码），思想一句话：**反复把语料中频率最高的相邻对合并成一个新符号**。我们用「cat / eat / fish」级别的玩具语料演示合并过程。假设语料只有三个词及其频次：cat 出现 3 次、eat 出现 2 次、fish 出现 2 次。初始词表是单个字符，每个词表示为字符序列：

- `c a t`（3 次）
- `e a t`（2 次）
- `f i s h`（2 次）

第 1 轮合并：统计所有相邻对的频次——`(c,a)` 出现 3 次，`(a,t)` 出现 $3+2=5$ 次，`(e,a)` 2 次，`(f,i)`、`(i,s)`、`(s,h)` 各 2 次。频率最高的是 `(a,t)`（5 次），合并成新符号 `at`。语料变成：

- `c at`（3 次）、`e at`（2 次）、`f i s h`（2 次）

第 2 轮合并：重新统计——`(c,at)` 3 次，`(e,at)` 2 次，其余 2 次。合并 `(c,at)` $\rightarrow$ `cat`：

- `cat`（3 次）、`e at`（2 次）、`f i s h`（2 次）

第 3 轮合并：现在 `(e,at)`、`(f,i)`、`(i,s)`、`(s,h)` 都是 2 次，打平时任取其一，比如合并 `(i,s)` $\rightarrow$ `is`：

- `cat`（3 次）、`e at`（2 次）、`f is h`（2 次）

第 4 轮合并：`(is,h)` 频次 2，合并 $\rightarrow$ `ish`：

- `cat`（3 次）、`e at`（2 次）、`f ish`（2 次）

真实 tokenizer 把这个过程跑几万轮，词表膨胀到 3 万到 15 万个符号，高频词整体成 token（`cat`），低频词拆成片段（`f` + `ish`）。工程含义有三点，都和钱有关：一是 token 是计费与算力的基本单位，同一个句子词表越大切得越短、训练越省；二是 tokenizer 的合并表必须和训练数据同分布——拿英文语料训的词表去切中文，一个汉字可能被拆成两三个 token，等于把中文成本凭空放大两三倍；三是词表大小直接决定 embedding 层参数量，词表 10 万、hidden 4096，仅输入 embedding 就是 $10^5 \times 4096 = 4.1 \times 10^8$ 个参数，这部分参数量要算进容量规划。

序列 packing：把短文档拼满 4096 个格子

训练时每个样本是固定长度的 token 序列，本书统一用 4096。问题来了：真实文档长度参差不齐，一条推文 30 个 token，一篇文档 800 个 token。如果一条序列只放一篇文档，剩下的格子怎么办？传统做法是 padding——塞特殊的填充符占位。但 attention 的计算量随格子数平方增长、FFN 随格子数线性增长，二者都不会跳过 padding 格子——**padding 位置照样消耗 FLOPs，却不产生任何学习信号**。想象你租了一辆 4096 个座位的巴士（一次矩阵乘），只坐了 800 人也要付全程油钱——这就是不 packing 的浪费。

packing 的做法是把多篇短文档首尾相接拼满 4096 个格子，文档之间用 EOS 隔开，并配合 attention mask 阻止跨文档互相注意（相当于巴士里加了隔板）。收益立竿见影：假设语料平均文档长 1024，不 packing 时有效 token 占比只有 $1024/4096 = 25\%$，等于 75% 的算力在白算；packing

后有效占比接近 100%，吞吐直接翻约 4 倍。工程上 packing 在离线 ETL 阶段完成（拼好再落盘），训练时零额外成本——这是数据管道里投入产出比最高的优化之一。

3.3 预训练：下一词预测，以及怎么盯 loss 曲线

预训练的训练目标简单到一句话：**给定前面所有 token，预测下一个 token**。第 2 章已经讲过，损失函数用交叉熵（第一册有完整推导），对整条序列求平均：

$$L = -\frac{1}{T}\sum_{t=1}^{T} \log P(x_{t+1} \mid x_1, \ldots, x_t)$$

其中 T 是序列长度（4096）。这个目标的魔力在于它不需要任何人工标注——文本本身就是标注，互联网就是题库。但工程上，预训练是一次数周甚至数月、数千卡的长跑，你不可能跑完才知道结果好坏。中途唯一的仪表盘就是 loss 曲线。学会读它，是训练工程师的基本功。

loss 曲线怎么读

一条健康的预训练 loss 曲线长这样：起步时（模型参数随机）loss 在 10 上下——词表 10 万时随机猜测的交叉熵约为 $\ln(10^5) \approx 11.5$，这是理论起点，开局远高于它就是 bug；然后快速下降，几千步内跌到 3 以下；之后进入漫长的缓慢下降期，以幂律的方式一点点逼近 2 以下。用后台的话类比：前段是"预热期 QPS 爬坡"，后段是"长尾优化"，收益递减但永不归零。

三种典型异常形态：

形态	曲线表现	常见原因	处置
正常下降	快速下降后平滑缓降，有微小毛刺	—	—
抖动/毛刺变密	下降趋势未变但尖峰频繁	脏数据 batch、某个数据源混入、梯度范数异常	定位 batch 来源；必要时跳过该 batch
发散/突跳（loss spike）	loss 突然暴涨甚至 NaN	学习率过高、数值溢出、硬件故障（坏卡算出脏梯度）	回滚到 spike 前 checkpoint，降 lr，跳过坏数据段重训

第 5 章会展开 spike 的处置 playbook，这里先建立一个意识：**盯 loss 曲线就像盯生产监控**——看趋势（下降速率是否符合预期）、看毛刺（尖峰频率与幅度）、看告警（NaN 直接PagerDuty）。成熟的训练框架会把 loss、梯度范数、学习率、吞吐画在同一块面板上，全天候有人值班。

学习率调度：warmup + cosine 的直觉

学习率 η 是训练全程最重要的超参，而它不是一个常数，是一条预设的曲线。工业标准形状是 **warmup + cosine decay**，用文字描述这条曲线：从 0 开始，用最初的两三千步**线性爬升**到峰值（比

如 3×10^{-4}），然后按住峰值附近短暂冲刺，最后按**余弦曲线缓缓下降**，到训练结束时降到峰值的约 10%。整条曲线像一次长跑的配速：先热身，再冲刺，最后减速进终点。

为什么是这种形状？三段各有理由：

- **warmup（热身）**：训练初期参数是随机的，梯度又大又不准；此时 Adam 的一阶、二阶动量估计也还没积累够样本，偏差很大。如果一上来就用峰值学习率，相当于让蒙眼的人全速冲刺，一步跨错就把参数甩进坏区域，后期很难救回来。线性热身给优化器几千步时间"校准仪表盘"。
- **高位平台（冲刺）**：中期参数已进入合理区域，用大学习率快速探索，下降最快。
- **cosine 衰减（减速）**：末期模型在最小值附近精调，步子太大只会在谷底两边来回震荡；余弦形状的好处是下降速率本身也是渐缓的——中期降得快（还在赶路），末期几乎走平（轻轻落位），比线性衰减的"最后一公里还在大步走"更稳。

经验法则：峰值 lr 随模型变大而变小（7B 常见 3×10^{-4} 量级，70B 降到一半以下），warmup 步数占总步数 0.1%–1%。定错峰值 lr 的后果两极分化：太小，跑完账单还没收敛；太大，spike 不断甚至发散重跑——两者都是真金白银。

3.4 SFT：教模型"听懂指令"，只在回答上算损失

预训练模型是"续写机器"：你写"中国的首都是"，它可能接"中国的首都是哪里？这是一个常见问题……"——它在模仿网页文体，而不是在回答你。SFT（Supervised Fine-Tuning，监督微调）用数十万条人工或机器生成的「指令-回答」对，把模型掰成"助手"形态。

工程上的核心机制只有一个：**loss mask**。训练时把问题和回答拼成一条序列喂进去，但只对回答部分的 token 计算损失，问题部分的标签被置为忽略值（PyTorch 惯例是 -100）。看一个小例子，样本是「问题：1+1=? 回答：2」，拼成序列并假设 tokenizer 切出如下 token：

位置	输入 token	预测目标（label）	是否计入损失
1	`<s>`	问	✗（mask 为 -100）
2	问	题	✗
3	题	：	✗
4	：	1	✗
5	1	+	✗
6	+	1	✗
7	1	=	✗
8	=	?	✗
9	?	答	✗

10	答	：	✗
11	：	2	✓
12	2	`</s>`	✓

12 个位置里只有最后 2 个计入损失：模型要学会的是"看到问题后，在位置 11 输出 2、在位置 12 输出结束符"。假设这两个位置的交叉熵分别是 0.7 和 0.1，这条样本的损失就是 $(0.7+0.1)/2=0.4$。

为什么要 mask？两个理由。一是**信号纯度**：问题的措辞千变万化，我们不想让模型浪费容量去"学会复述问题"，只想让它学会"给定问题生成好回答"——mask 相当于告诉优化器，前 10 个位置的预测误差不计入 KPI。二是**推理一致性**：线上服务时，问题由用户输入、模型只需生成回答，训练时只对回答算损失恰好对齐了这个使用方式。工程含义：SFT 数据管道的每条样本除了 token ID，还要带一条等长的 mask 数组；多条短样本同样可以 packing 进 4096 格子，但 mask 边界必须和文档边界一起拼，拼错一格，损失就算错了地方——这类 bug 不报错、不崩溃，只是模型悄悄学歪，是 SFT 管道里最难排查的问题之一。

3.5 对齐：RLHF 与 DPO 的工程取舍

SFT 之后的模型会回答问题了，但不知道"哪个回答更好"。对齐阶段用人类偏好数据继续调教。方法论（偏好建模、PPO、DPO 的数学推导）已在**第二册第 7 章**完整展开，本节只回答分布式工程师关心的三个工程问题：奖励模型怎么训、PPO 为什么贵、DPO 为什么流行。

奖励模型（RM）怎么训：收集同一个提示下的多个回答，让人工（或强模型）排序，得到"回答 A 优于回答 B"的成对数据；然后拿 SFT 模型当底座，把输出头换成分数头，训练它给好回答打高分、坏回答打低分。成本构成：一是偏好标注数据（数万条起，优质人工标注单价高，是主要成本）；二是训练本身只比一次小规模 SFT 略贵，GPU 账单不是大头。**RM 的成本瓶颈在数据，不在算力。**

PPO 为什么贵：PPO 在线强化学习阶段，显存里要同时养四个模型——正在训练的 policy、冻结的 reference（防跑偏基准）、打分的 reward model、估计优势的 value model（critic）。按附录 A2 的口径，7B bf16 权重一份是 $7\times10^{9}\times2\ \mathrm{B}=14$ GB，四个模型光权重就约 56 GB，而 policy 和 critic 还要训练，各自背上优化器状态；除此之外，PPO 每轮要在线采样——policy 现生成回答、RM 现打分、再更新——这是一次"训练套着推理"的混合负载，吞吐被生成速度拖住，调度复杂度远超普通训练。账面上的结论：PPO 的综合成本（显存、卡时、工程复杂度）是 SFT 的数倍。

DPO 为什么流行：DPO 把"训奖励模型 + 在线强化学习"两步折叠成一个闭式损失，直接在偏好对上做监督式训练。工程后果非常诱人：只需 policy 和 reference **两个模型**（reference 还可以用低精度冻结副本），无在线采样、无 RM 服务、无四个模型的显存挤压，训练循环长得和普通 SFT 几乎一样，现成管道改改就能跑。代价是效果上限略逊于调好的 PPO、对偏好数据分布更敏感。取舍结论与后台世界如出一辙：PPO 是"性能上限更高的分布式事务"，DPO 是"够用、便宜、好运维的最终一致性"——所以社区默认先用 DPO 拿到 80 分，确有差距再回头啃 PPO。

3.6 一个 token 的旅行：数据管道工程师视角

最后，跟随一个 token 走完全程，把本章串起来。假设它出生于某个论坛帖子，最终成为 7B 模型一次梯度更新中参与计算的百万分之一的损失。

第一站：对象存储。 爬虫把它连同几亿个网页写进对象存储（S3/OSS），原始 HTML，PB 级。对象存储的角色和大数据时代一样：便宜、海量、吞吐尚可、延迟无所谓——训练数据是"带宽敏感、延迟不敏感"负载的典型。

第二站：ETL 集群。 Spark/Ray 作业把它从 HTML 里抠出来（正文提取），切 shingle、算 MinHash 签名、被判重——它的三个克隆兄弟在这一站被删除，它活了下来；再过质量过滤器、按语种和来源打标签、按配比表分桶。然后被 BPE 切成一串 token ID，和几十篇短文档 packing 进同一个 4096 格子，落盘为列式文件（Parquet/Arrow 分片，每片约 1 GB，便于并行读和断点续训时按片跳过）。

第三站：训练节点的本地缓存。 训练集群开跑前，热数据分片被预取到各节点 NVMe 本地盘——对象存储的聚合带宽撑不起 1024 卡同时随机读，本地缓存层把"读训练数据"变成"读本地盘"，这和给数据库加 Redis 是同一个思想，只是缓存的是分片文件。

第四站：DataLoader 多进程预取。 每个训练进程挂一个 DataLoader，若干 worker 子进程并行地：读分片 $\rightarrow$ 按全局种子 shuffle 采样 $\rightarrow$ 组 batch $\rightarrow$ pin 到锁页内存。预取队列深度（prefetch factor）就是管道里的 buffer：深度不足，GPU 算完一步要干等数据（GPU 利用率掉、MFU 掉）；深度过大，内存被撑爆。目标只有一个——**让 H2D 拷贝永远赶在计算前面**。每步消耗约 $4096 \times \text{全局 batch}$ 个 token，数据管道的吞吐必须稳定压过这个消耗速率，这就是它的 SLO。

第五站：GPU 显存。 batch 经 PCIe 拷贝进 HBM，我们的 token 躺在其中一个格子里。前向时它被 embedding 成向量、流过 32 层 Transformer；反向时它贡献一份微小的梯度，AllReduce 进全局梯度，推动 70 亿个参数挪动一小步——第 2 章解剖的那个 $w' = w - \eta \frac{\partial L}{\partial w}$，此刻在万亿倍的规模上发生。两万亿个 token 各走一遍这样的旅程，加起来就是那 8.4×10^{22} 次浮点运算，和一个"读懂了互联网"的模型。

至此，从原始网页到可用模型的全流程已经走完。接下来的问题转向集群内部：这 70 亿参数和 112 GB 的显存账，一张 80 GB 的卡根本放不下——怎么办？这就是第 4 章分布式并行的主题。

第 4 章 分布式并行：把 70 亿参数放进集群

前两章解决的是「一个 GPU 上怎么训」的问题。但只要你把第 2 章的显存账认真算一遍，就会发现一个残酷的事实：7B 这样的「小模型」，训练时需要的显存已经远超单张 H100 的 80 GB。这一章我们做一件分布式工程师最熟悉的事——**分片与副本**：把模型、数据、优化器状态切开放进集群，用通信换显存，用并行换时间。数据并行（DP）、张量并行（TP）、流水线并行（PP）、ZeRO 这四种手段，本质上是你在分库分表、MapReduce、微服务拆分里早已用过的思想，只是切的对象从「数据行」变成了「矩阵和层」。

4.1 显存账：为什么单卡放不下

先建立直觉：训练一个模型，显存里要同时住着好几类人——模型权重本人、权重的「工作副本」、每个参数的梯度、以及优化器为每个参数记的两本「小账」。再加上前向传播留下的中间结果（激活值）。这些人一个都少不了。

类比一下：这像你维护一个有状态服务，除了数据本身（权重），还要存 binlog（梯度）、两份副本状态（优化器动量）、以及请求处理过程中的上下文（激活值）。只算「数据本身多大」是新人最容易犯的容量规划错误。

手算（统一算例 A2）：7B 模型，$N = 7 \times 10^9$ 个参数，采用混合精度 AdamW 训练（第 5 章会细讲混合精度，这里先用结论）。每个参数需要：

显存项	精度	每参数字节数	7B 总量
fp32 主权重（更新用真身）	fp32	4 B	$7 \times 10^9 \times 4 = 28$ GB
fp32 梯度	fp32	4 B	28 GB
一阶动量 m（梯度的滑动平均）	fp32	4 B	28 GB
二阶动量 v（梯度平方的滑动平均）	fp32	4 B	28 GB

合计每参数：

$$4 + 4 + 4 + 4 = 16 \text{ B}/\text{参数}$$

$$7 \times 10^9 \times 16 \text{ B} = 112 \times 10^9 \text{ B} = 112 \text{ GB}$$

这还没完。混合精度训练中前向/反向实际跑的是 bf16 副本，还要再加一份 bf16 权重：

$$7 \times 10^9 \times 2 \text{ B} = 14 \text{ GB}$$

总计约 $112 + 14 = 126$ GB。正文我们统一采用保守口径：**约 112 GB（不计激活值）**。

激活值另计，而且不容小觑：反向传播要用到每一层的中间输出，batch 越大、序列越长，激活值越大（第 5 章的梯度检查点就是专门对付它的）。经验上，7B 训练时激活值轻松达到几十 GB。

结论一目了然：单卡 80 GB，光「16 B/参数」这条就爆了，加上激活值更是无望。**显存墙是第一性的约束**，它决定了我们必须并行。接下来的所有方案，都是在回答同一个问题：112 GB 怎么切、切了之后怎么同步。

4.2 数据并行 DP：最简单也最常用的一刀

直觉：模型放不下，但数据可以切。每张卡都放一份完整模型（副本），把 batch 切成 K 份，每张卡算自己那份数据的梯度，最后大家把梯度「对齐」成一致的，再各自更新。这就是数据并行（Data Parallelism, DP）。

类比：这就是 MapReduce。map 阶段各 worker 处理自己的数据分片；reduce 阶段把局部结果聚合为全局结果。DP 里每张卡是 mapper（算局部梯度），AllReduce 是 shuffle + reduce（求全局平均梯度）。区别只在于 MapReduce 的 reduce 通常有中心节点，而 AllReduce 是无中心的——这正是第 1 章介绍的集合通信原语。

数学：设全局 batch 为 B，K 张卡各取子 batch B/K，各自前向+反向得到局部梯度 g_k。AllReduce 后每卡得到：

$$g = \frac{1}{K}\sum_{k=1}^{K} g_k$$

这与单卡跑完整 batch B 在数学上等价（梯度是线性平均）。所以 DP 不改变训练语义，只改变吞吐——这是它成为首选的原因。

通信量手算：每步要同步的数据量 = 全部梯度的大小。按 bf16 梯度口径：

$$S = 7 \times 10^9 \times 2\ \text{B} = 14\ \text{GB}$$

代入第 1 章的 ring AllReduce 时间公式（K 张卡、带宽 B_w）：

$$T \approx 2 \times \frac{K-1}{K} \times \frac{S}{B_w}$$

情形一：64 卡跨机 DP，机间带宽 50 GB/s（A3）：

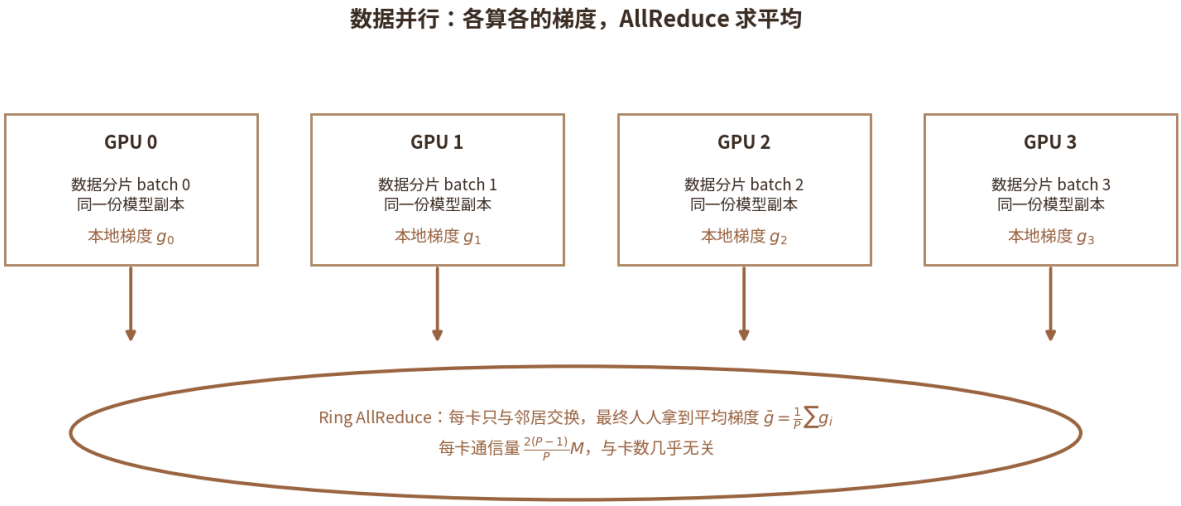
$$T \approx 2 \times \frac{63}{64} \times \frac{14}{50} \approx 2 \times 0.28 \approx 0.55\ \text{s}$$

情形二：8 卡机内 DP，NVLink 900 GB/s：

$$T \approx 2 \times \frac{7}{8} \times \frac{14}{900} \approx 0.027\ \text{s}$$

差 20 倍。每一步训练多花半秒钟等梯度，7B 训 2T tokens 要几十万步，这就是天文数字的差距——**网络拓扑直接决定并行策略**，这句话会在本章反复出现，也是第 8 章的主题。

工程含义：DP 的优点是简单、语义透明、扩展性好；缺点有两个——一是每卡都要存完整的一份 112 GB，显存零节省（所以 DP 从来不能单独用）；二是每步一次 14 GB 的同步，通信开销随卡数与带宽博弈。第 5 章会讲怎么把通信藏在反向计算后面。



4.3 张量并行 TP：把矩阵从中间切开

直觉：DP 解决不了「单层太大」的问题。张量并行（Tensor Parallelism, TP）的思路是：一个巨大的矩阵乘法，可以由多张卡各算一部分再拼起来——矩阵乘法天然可分块。

类比：大表 join 时的分桶：两张表都按 join key 哈希到不同节点，各节点算局部 join，再合并结果。TP 就是在线性代数里做这件事。

数学：以一层线性变换 $Y = XW$ 为例。X 是 $b \times h$ 的激活，W 是 $h \times h'$ 的权重。两种切法：

切法一：按列切 W。把 W 沿输出维度切成两半 $W = [W_1 \mid W_2]$，卡 0 拿 W_1、卡 1 拿 W_2：

$$Y_1 = XW_1, \quad Y_2 = XW_2, \quad Y = [Y_1 \mid Y_2]$$

每张卡拿到完整的 X，各算输出的一半列，最后**拼接**（AllGather）即可。全程一次通信，而且如果下一层恰好按行切，连拼接都省了——这是 Megatron 的经典设计。

切法二：按行切 W。把 W 沿输入维度切，同时把 X 也按列切成对应的两半 $X = [X_1 \mid X_2]$：

$$Y = XW = X_1W_1 + X_2W_2$$

每张卡算出的是**部分和**，需要一次 AllReduce 相加才得到真正的 Y。

手算小例子：设 $X = [1, 2]$（1×2），$W = \begin{pmatrix} 3 & 5 \\ 4 & 6 \end{pmatrix}$。按列切：卡 0 拿第一列 $[3, 4]^T$，算 $Y_1 = 1 \times 3 + 2 \times 4 = 11$；卡 1 拿第二列，算 $Y_2 = 1 \times 5 + 2 \times 6 = 17$。拼接得 $Y = [11, 17]$。验证：$XW = [1 \times 3 + 2 \times 4,\ 1 \times 5 + 2 \times 6] = [11, 17]$。分毫不差。

Transformer 一层的通信次数：一层 Transformer 有两大块——注意力（Q/K/V/O 四个投影）和 MLP（升维 + 降维两个矩阵）。Megatron 的标准切法是「按列切接按行切」成对出现：Q/K/V 和升维矩阵按列切（免通信），O 投影和降维矩阵按行切（各需 1 次 AllReduce）。所以 **TP=2 时每层 2 次 AllReduce**（反向还有一次，量级翻倍，这里只数前向）。注意：AllReduce 的对象不是权重，而是**激活值**。

算一笔账（为什么 TP 必须留在机内）：设每层每次 AllReduce 的激活约 32 MB（例如 4096 个 token、hidden 4096、bf16：$4096 \times 4096 \times 2$ B $= 32$ MB）。7B 有 32 层，每层 2 次，前向一遍每张卡的通信量：

$$32 \text{ MB} \times 2 \times 32 \text{ 层} = 2 \text{ GB}$$

- 机内 NVLink（900 GB/s，ring 有效带宽打八折约 720 GB/s）：$2 / 720 \approx 2.8$ ms——可以忍。
- 跨机 IB（50 GB/s）：$2 / 50 = 40$ ms——前向一遍光等 TP 通信就 40 ms，而单层计算本身才毫秒级，GPU 会大面积空转。

$900 / 50 = 18$ 倍的带宽差距，把 TP 死死锁在 NVLink 域内：**TP 一般只在机内 8 卡之间做**。这是本章第一条经验法则。

4.4 流水线并行 PP：按层切，像工厂流水线

直觉：模型按层竖着切，第 0–7 层放节点 0，第 8–15 层放节点 1……数据像流水线上的工件，流完一层流下一层。这就是流水线并行（Pipeline Parallelism, PP）。

类比：微服务调用链——订单服务调库存服务再调支付服务。区别在于微服务各处理不同请求，而 PP 处理的是同一个 batch 的不同阶段。

bubble 的产生（手算）：问题出在哪里？训练一个 step 要先前向再反向，而反向必须等前向流完才能从末端开始。设流水线级数 $P = 4$，把一个 batch 切成 $M = 4$ 个 micro-batch 依次灌入。以「前向一格、反向一格」为单位时间，画时间轴：

- 第 1 格：级 1 做 mb1 前向，级 2/3/4 空闲；
- 第 2 格：级 1 做 mb2，级 2 做 mb1；
- ……前向要 $P + M - 1 = 7$ 格才能全部流完，反向同样要 7 格……

总时长为 $P + M - 1$ 个「级 × micro-batch」单位时间（以充能排空模型计），而真正干活的只有 M 个。空闲比例（bubble 占比）为：

$$\text{bubble} = \frac{P-1}{M+P-1}$$

代入 $P=4$，$M=4$：

$$\frac{3}{4+3} = \frac{3}{7} \approx 43\%$$

将近一半的算力在空转。把 micro-batch 增到 $M=8$：

$$\frac{3}{8+3} = \frac{3}{10} = 30\%$$

M 越大 bubble 越小，但 micro-batch 变小会拉低单卡计算效率、增大激活份数（显存压力），这是个 trade-off。

1F1B 一句话直觉：朴素调度（GPipe）是「先把所有 micro-batch 前向跑完，再统一反向」，代价是要同时缓存全部 M 份激活。1F1B（one-forward-one-backward）让每个级进入稳态后交替做「一个前向、一个反向」，峰值只需缓存约 P 份激活，显存大降而 bubble 不变——用调度顺序换显存，工程上几乎总是赚。

PP 的通信：相邻级之间只传边界激活（point-to-point send/recv），每 micro-batch 每层边界一次，数据量小、频率低，对带宽不敏感。所以 **PP 是三种并行里最敢跨机的**——第二条经验法则。

流水线并行的气泡：4 级流水 × 8 个 micro-batch（利用率 = 8/11 ≈ 73%）



空格 = 气泡（GPU 空转）。气泡占比 $\frac{P-1}{M+P-1}$：micro-batch 越多气泡越小

4.5 ZeRO：把「每卡一份」的冗余削掉

直觉：回到 4.2 的痛点——DP 下 64 张卡，每张都存一模一样的 112 GB，63 份是纯浪费。ZeRO（以及 PyTorch 的 FSDP）问了一个分布式工程师会觉得理所当然的问题：**副本既然内容相同，为什么不只存 1/64，用时再取？** 这就是参数分片，和你给缓存做 sharding 是同一个动作。

三个阶段，各分片什么（以 7B、64 卡 DP 为例，先回顾每卡基线：fp32 主权重 28 GB + 梯度 28 GB + m 28 GB + v 28 GB = 112 GB，另有 bf16 权重副本 14 GB）：

阶段	分片对象	每卡显存（64 卡）	账
基线 DP	不分片	$112 + 14 = 126$ GB	全量副本
ZeRO-1	optimizer 状态（m、v、fp32 主权重，共 84 GB）	$84/64 + 28 + 14 \approx 43$ GB	优化器状态分 64 份
ZeRO-2	+ 梯度（28 GB）	$112/64 + 14 \approx 15.8$ GB	梯度也分片
ZeRO-3	+ 参数本身（bf16 权重 14 GB）	$112/64 + 14/64 \approx 1.97$ GB	全部分片

ZeRO-3 的数字值得停下来看一眼：每卡约 2 GB 的模型状态，加上激活值，一张 80 GB 的卡不仅能训 7B，还能训 70B。**显存从「每卡全量」变成「每卡 $1/K$」**，这就是 ZeRO 的全部秘密。

代价是通信：分片意味着用的时候没有。

- 前向/反向要用某层权重？先 **AllGather** 把该层的参数分片从 64 张卡收齐，用完即弃（ZeRO-3）；
- 反向算完梯度？用 **ReduceScatter** 替代 AllReduce——每张卡只归约并保留自己负责的那 $1/64$ 梯度分片（ZeRO-2/3）；
- 更新时各卡只更新自己那 $1/64$ 的 fp32 主权重与动量（ZeRO-1 起）。

值得记住的结论：**ZeRO-1/2 的通信量和普通 DP 完全一样**（ReduceScatter + AllGather 合起来等价于一次 AllReduce 的传输量），白捡显存；ZeRO-3 因为每层都要取参数，通信量约增至 1.5 倍。这是典型的「通信换显存」，和第 5 章「时间换显存」的梯度检查点互为镜像。

4.6 组合：3D 并行的经验法则

单一手段都不够：DP 不省显存，TP 出不了机，PP 有 bubble。真实训练是三者（加 ZeRO）的组合。经验法则一句话：**TP 机内、PP 跨节点、DP 最外层**。

理由串起来就是本章前面的账：

1. **TP 通信最频繁**（每层 2 次 AllReduce，对象是激活），必须吃 NVLink 的 900 GB/s，限定在机内 8 卡；
2. **PP 通信最少**（级间只传边界激活），放跨节点，吃 50 GB/s 的 IB 也够用；
3. **DP 在最外层**，每步一次梯度 AllReduce，通过 ZeRO 顺手把显存也分了。

示例：1024 卡训 7B。1024 卡 = 128 节点 × 8 卡。配置 **TP=8 × PP=2 × DP=64**，验证：$8 \times 2 \times 64 = 1024$。

- TP=8：恰好填满一个节点的 NVLink 域，一层 Transformer 的所有矩阵切 8 份，通信不出机；
- PP=2：模型切两段，每段 16 层，bubble 为 $(2-1)/(M+1)$，$M=16$ 时仅约 6%；段间点对点传激活，跨机无所谓；
- DP=64：64 个「TP8×PP2 组」（每组 16 卡，恰好 2 节点）各吃一份数据，配合 ZeRO-1/2，每卡模型状态从 126 GB 降到十几 GB，激活值随便放。

为什么不是 TP=4 × PP=4 × DP=32？ 乘积同样是 1024，但两处吃亏：PP=4 的 bubble 按 $M=16$ 算是 $3/18\approx 17\%$，比 PP=2 的 6% 差了近两倍；TP=4 又浪费了机内另外 4 条 NVLink 通道的切分能力。反过来 TP=8 × PP=1 行不行？PP=1 意味着不做流水，模型必须整份放进「TP=8 组」——7B 的 112 GB 摊到 8 卡是 14 GB/卡，放得下，但这样 DP 维度变成 128，跨机 AllReduce 组更大、对 straggler 更敏感，而且失去了用 PP 把激活值沿层分摊的好处。真实选型就是这样在 bubble、显存、通信半径三者之间找平衡点。

这个配置没有一处浪费：快的互联留给话最多的 TP，慢的互联给话最少的 PP/DP，显存冗余交给 ZeRO。**并行策略的本质，是把通信模式和网络拓扑做对齐**——这句话建议抄在工位上。

4.7 概念对照：你早就认识它们

收尾给一张迁移表。分布式并行里的每个概念，都能在你的老本行里找到对应物：

分布式后台概念	训练并行中的对应物	说明
分片（sharding）	TP（切矩阵）/ PP（切层）/ ZeRO（切状态）	按不同维度切：行、列、层、优化器状态
副本（replication）	DP	每卡一份完整模型，切的是数据
一致性协议	梯度同步语义	AllReduce 平均 = 每步强一致；异步 SGD = 最终一致（几乎没人用）
无状态化/分片再均衡	ZeRO 的 AllGather/ReduceScatter	用时取回、用完即弃
流水线/调用链编排	PP 调度（GPipe、1F1B）	bubble = 流水线充能排空损耗
容错与恢复	checkpoint + 弹性训练	第 5 章展开：千卡集群按小时出故障
MapReduce	DP 的 map（局部梯度）+ reduce（AllReduce）	shuffle 阶段由 ring 算法无中心化完成

值得强调的是「一致性」这一行：同步训练里，所有副本每步通过 AllReduce 对齐梯度，语义上是**强一致**——任何一卡慢一步（straggler），全队等它。这就是为什么训练集群对抖动、对慢节点如此敏感，也是第 5、8 章监控与容错设计的出发点。

本章小结：112 GB 的显存墙逼出了并行；DP 切数据、TP 切矩阵、PP 切层、ZeRO 切冗余状态；组合时让通信频率与带宽层级对齐。下一章我们进入训练工程的细部：混合精度、稳定性、以及怎么让这 1024 张卡的钱花得值。

第 5 章 训练工程：混合精度、稳定性与省钱

第 4 章解决了「放得下」的问题：靠 3D 并行把 70 亿参数和 112 GB 的优化器状态摊到 1024 张卡上。但放得下只是起点，真正烧钱的是接下来三个问题：**算得快不快**（每个 bit 的精度能不能砍）、**跑得稳不稳**（loss 突然跳起来怎么办）、**钱花得值不值**（1024 张卡的峰值算力，实际用出来了几成）。本章把这三件事拆开，每一件都给你能上手算的数字和能直接照搬的 playbook。

这一章的思维方式你其实早就有了：混合精度是「编码层面的位预算分配」，梯度裁剪是「单步变更的限流器」，loss spike 处置是「熔断加回滚」，梯度检查点是「缓存逐出后回源重算」，通信重叠是「异步批量发送」，checkpoint 是「备份与 binlog 位点」，MFU 是「资源利用率 SLO」。大模型训练没有发明新的工程问题，只是把它们的量级推到了极端。

5.1 混合精度：浮点数的位预算怎么花

直觉：一张固定大小的床，量程和分辨率只能选一个

计算机里的浮点数就是二进制版的科学计数法。一个浮点数由三部分拼成：

$$\text{值} \approx (-1)^{s} \times 2^{e-\text{bias}} \times (1 + f)$$

其中 s 是**符号位**（1 位，表示正负），e 是**指数位**（决定 2 的多少次幂），f 是**尾数位**（决定有效数字）。总位数是固定的预算：FP32 有 32 位，FP16 和 BF16 各只有 16 位。预算怎么切，决定了这个格式的性格：

- **指数位多** → 能表示的数范围大（天文数字和尘埃数字都装得下），类似 int64 之于 int32 的「量程」；
- **尾数位多** → 有效数字多、分辨率高，相邻两个可表示的数之间的缝隙小。

三种格式的位结构对比

格式	符号位	指数位	尾数位	表示范围（约）	十进制有效位数（约）
FP32	1	8	23	$\pm 3.4 \times 10^{38}$	7 位
FP16	1	5	10	± 65504	3 位
BF16	1	8	7	$\pm 3.4 \times 10^{38}$	2~3 位

注意 BF16 的设计哲学：它把 FP32 的 32 位直接「砍尾」——指数位原样保留 8 位，尾数从 23 位砍到 7 位。所以 BF16 的**范围和 FP32 一模一样**，只是精度粗。而 FP16 走的是另一条路：指数位砍到 5 位换尾数 10 位，精度稍好但范围急剧缩水。这就像监控系统的指标编码：你可以选「量程覆盖 ±10^38 但只保留 2 位有效数字」，也可以选「精度到小数点后 10 位但超过 6 万就溢出」。

手算：为什么 10^{-8} 的梯度在 FP16 里直接消失

FP16 的指数位有 5 位，bias $= 2^{5-1} - 1 = 15$。最小**正规格数**（normal number，即尾数前导为 1 的数）的指数是 $e = 1$，对应：

$$2^{1-15} = 2^{-14} = \frac{1}{16384} \approx 6.1 \times 10^{-5}$$

也就是说，任何绝对值小于 6.1×10^{-5} 的数，在 FP16 里要么靠「非规格数」勉强凑合（精度极差），要么直接被截成 0。

现在看训练现场。深度网络靠近输入端的层，梯度经过几十层链式法则连乘，量级到 10^{-8} 非常常见。而：

$$10^{-8} \approx 2^{-26.6} \ll 2^{-14}$$

这个梯度写进 FP16 的瞬间就变成了 0。梯度为 0，参数不更新，这部分网络等于停止了学习——这就是所谓的**梯度下溢**（underflow）。

FP16 的解法：loss scaling。 思路朴素得像凑满减：既然梯度太小，那就先把损失放大。具体地，前向算完损失 L 后，先乘一个缩放系数 S（常取 $2^{14} \approx 16384$，或动态调整），再反向传播。链式法则是线性的，每个梯度都自动跟着放大 S 倍：

$$10^{-8} \times 2^{14} \approx 1.6 \times 10^{-4} > 6.1 \times 10^{-5}$$

放大后的梯度稳稳落在 FP16 的正规格数范围内。等梯度传到优化器手里、准备更新参数之前，再除以 S 缩回去，更新量分毫不差。工程上常用**动态 loss scaling**：定期检查梯度有没有溢出（FP16 上溢到 Inf），溢出就把 S 减半并跳过这一步更新，一段时间没溢出就把 S 翻倍，自动找到当前训练阶段能用的最大缩放。

BF16 为什么不需要这套杂技？ 因为 BF16 的指数位和 FP32 一样是 8 位，最小正规格数约 $2^{-126} \approx 1.2 \times 10^{-38}$。$10^{-8}$ 的梯度在 BF16 里舒舒服服，不存在下溢问题。代价是尾数只有 7 位：两个相邻可表示数的相对缝隙约 $2^{-7} \approx 0.8\%$。这对前向反向的中间计算无所谓，但对**参数更新**是致命的——学习率 η 乘梯度后，更新量往往只有参数本身的万分之一，$w + \Delta w$ 在 BF16 里经常被「四舍五入」回 w 自己，小步更新被悄悄吞掉。

工程含义：混合精度的标准配方

所以工业界的标准做法是**混合精度**，各取所长：

- **前向和反向计算用 BF16**：Tensor Core 对 16 位格式的吞吐是 FP32 的数倍，激活显存也减半；
- **优化器保留一份 FP32 主权重**：更新先在 FP32 上做（小步长不会被吞），每步再 cast 一份 BF16 副本给下一步前向用；

- 梯度、一阶动量、二阶动量也都留 FP32。

这正是附录 A2 那笔账的由来：每个参数 4 B 主权重 + 4 B 梯度 + 4 B 一阶动量 + 4 B 二阶动量 = 16 B，7B 模型光优化器状态就约 112 GB，外加 BF16 前向副本约 14 GB。混合精度不是「为了省显存」，而是「用 FP32 守住数值正确性，用 BF16 买到 16 位格式的算力和带宽」——省下来的显存只是顺手的红利。



5.2 梯度裁剪：给单步更新装一个限流器

直觉与类比

训练是下山，但 loss 曲面不是光滑的碗，而是到处是悬崖的沟壑。绝大多数步梯度都很温和，偶尔踩到一处陡坡，梯度突然比平时大几十倍——如果照单全收，这一步会直接跨过谷底冲上对面的山坡，前面几万步的进展付诸东流。

这在后台系统里有精确对应物：**限流器**。正常流量随便过，突发流量按上限削平，保护下游不被一波带走。梯度裁剪就是参数更新的限流器。

数学与手算

做法是对**所有参数的梯度拼成的全局向量** g 计算 L_2 范数：

$$\|g\| = \sqrt{\sum_i g_i^2}$$

若 $\|g\|$ 超过阈值 τ（工程上常取 1.0），就整体等比缩小：

$$g \leftarrow g \cdot \frac{\tau}{\|g\|}$$

注意是**等比缩放**，方向完全不变，只改步长；也不是逐参数截断，而是全局一个系数，保证各参数更新的相对比例不被破坏。

手算例子。 假设某一步只有两个梯度分量 $g = (3, 4)$，阈值 $\tau = 1.0$：

1. 范数：$\|g\| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$；
2. $5 > 1.0$，触发裁剪，缩放系数 $\frac{\tau}{\|g\|} = \frac{1.0}{5} = 0.2$；
3. 裁剪后：$g' = (3 \times 0.2,\ 4 \times 0.2) = (0.6,\ 0.8)$；
4. 验算：$\|g'\| = \sqrt{0.6^2 + 0.8^2} = \sqrt{0.36 + 0.64} = 1.0$。

方向 $(3, 4)$ 和 $(0.6, 0.8)$ 完全一致（都是「3-4-5 直角三角形」的那条边），只是模长被压到阈值以内。

工程含义

三条经验：第一，阈值 1.0 是行业惯例起点，一般不需要精细调；第二，**梯度范数曲线是最重要的训练健康指标**，它的尖峰永远早于 loss 尖峰出现——下一节的 loss spike 预警就靠它；第三，裁剪发生在 AllReduce 之后、优化器更新之前，分布式下要保证各卡算的是同一个全局范数，否则各卡缩放系数不一致就破坏了同步语义。

5.3 loss spike：训练中的「线上事故」与处置 playbook

现象

loss spike 是长训最经典的事故：loss 曲线平稳下降了几周，突然在几步之内跳升 2 到 10 倍。运气好，几十步后自己回落；运气差，loss 卡在高位再也回不去，模型「学坏了」；最糟的情况是直接 NaN——某个中间值溢出成 Inf，经过一层层运算污染所有梯度，训练彻底报废。看监控时你会发现梯度范数几乎总是在同一时刻出现一根尖刺，这就是 5.2 的裁剪器没能完全压住的陡坡。

类比：熔断 + 回滚的 oncall runbook

这套事故和后台的错误率尖刺完全同构：监控告警（loss/grad norm 尖峰）→ 止血（熔断、回滚到上一个稳定版本）→ 定位（找到引入问题的变更/流量）→ 恢复（灰度放量观察）。训练侧的对应物是：checkpoint 就是你的「上一个稳定版本 + binlog 位点」，可疑数据就是「毒消息」（poison message），临时降学习率就是「降级预案」。

常见原因

原因类别	典型表现	排查手段
数据脏	spike 时刻某个 batch 含乱码、超长重复串、混入的 HTML/代码	回放 spike 前后的 batch 样本，检查数据 id
学习率过高	warmup 不足或调度配置错误，spike 从训练早期就频繁出现	对照 lr 曲线与 spike 时刻

硬件故障	某卡显存 bit flip、网络丢包导致 AllReduce 结果错误	查 XID 错误、ECC 计数、NCCL 日志
数值溢出	FP16 训练 loss scale 过大，梯度上溢成 Inf	查动态 loss scaling 的溢出计数
数据配比突变	换了一批数据后分布突变，loss 系统性上移且不回	检查数据混配开关与版本切换点

处置 playbook

1. **发现**：对 loss 和梯度范数设滑动窗口告警（如超过过去 1000 步均值的 3 倍即告警），不要等人工盯屏；
2. **止血**：暂停训练，**回滚到 spike 发生前的最近一个 checkpoint**——这是最关键的一步，spike 之后保存的 checkpoint 都可能已带「内伤」；
3. **定位与隔离**：找到 spike 时刻对应的 batch，**跳过这段可疑数据**继续训练（data skipping），就像消息队列里跳过毒消息；
4. **降级**：恢复时临时把学习率减半、把梯度裁剪阈值收紧，平稳度过这段区域后再恢复原计划；
5. **查硬件**：如果找不到数据原因，按上表查硬件日志，坏卡直接踢出资源池；
6. **留现场**：spike 前后的 loss、梯度范数、数据 id、节点列表全部归档，相当于保留 core dump，事后复盘用。

工程含义

playbook 里每一步的有效性都取决于一个前提：**你有足够近、足够干净的 checkpoint 可以回滚**。回滚损失 = checkpoint 间隔 × 训练速度。如果每 8 小时存一次，一次 spike 最多白烧 8 小时 × 1024 卡时；如果每 2 小时存一次，损失直接除以 4。这就是 5.6 要算的账。

5.4 梯度检查点：用 1/3 的时间换 3 倍的显存

直觉与类比

反向传播需要前向传播的中间激活值做原料（第 2 章的链式法则，每一步都要乘上前向时的局部导数）。默认做法是把 L 层所有激活全存下来，反向时随取随用——显存占用 $O(L)$，长序列、大模型下这笔激活账动辄几十 GB，比权重还凶。

梯度检查点（gradient checkpointing）的思路是**缓存逐出、回源重算**：只存每层的入口激活，反向算到某一层时，从该层入口把这一层的前向重跑一遍，临时恢复出中间激活，用完即扔。这和数据库的「checkpoint + WAL 重放」是同一个思想：不存全量现场，只存锚点，需要时从锚点重放。

账：时间与显存的交换

时间账。 由第 2 章的 $6ND$ 推导知道，反向的计算量约是前向的 2 倍。设一次前向耗时 F，反向耗时 $B = 2F$：

- 不开启检查点：每步 $F + B = 3F$；
- 开启检查点：反向时每层要先重算一次前向（$+F$），再做原本的反向：每步 $F + (F + B) = 4F$。

$$\frac{4F - 3F}{3F} = \frac{1}{3} \approx +33\%$$

即**训练时间增加约三分之一**。

显存账（手算）。 假设模型有 $L = 36$ 层，每层激活 1 份。不存检查点要存 36 份。分段策略：把 36 层切成 $\sqrt{36} = 6$ 段，每段 6 层，只存 6 个段入口作为锚点；反向处理某段时，从锚点重算该段，段内最多临时占 6 份。峰值占用约 $6 + 6 = 12$ 份，只有原来的三分之一。一般地，锚点取 $\sqrt{L}$ 个时峰值约 $2\sqrt{L}$，激活显存从 $O(L)$ 降到 $O(\sqrt{L})$ 级别；极端做法只留一个入口，接近 $O(1)$，但重算次数变多、时间代价失控，所以工程上取 $\sqrt{L}$ 这个甜点。

工程含义

+33% 时间换激活显存数倍下降，几乎总是划算的：省下的显存可以换更大的 micro-batch、更长的序列，吞吐往往反而提升。这也是为什么 FlashAttention（第 7 章）内部同样用「不存中间矩阵、反向重算」来压显存——重算是整个大模型系统里最常用的通用货币。唯一的例外是显存本来就不紧张的小模型，白烧 33% 时间没有任何收益。

5.5 通信计算重叠：把 AllReduce 藏进反向传播

直觉与类比

第 4 章讲过数据并行的瓶颈：每步结束要对所有卡的梯度做 AllReduce，7B 模型 bf16 梯度 14 GB，走 50 GB/s 的机间网络裸传就要约 0.28 秒。如果老老实实「算完再传」，这 0.28 秒里 1024 张卡集体空转。

但反向传播有个结构性红利：**它是从最后一层往前算的**。第 L 层的梯度最先算完，此时前面 $L - 1$ 层的计算还要持续很久。既然如此，第 L 层的梯度一出炉就立刻异步发出去做 AllReduce，通信和后续层的计算同时进行——这就是通信计算重叠（overlap）。类比后台：Kafka producer 的批量异步发送，消息一进 buffer 满一桶就 flush，主流程不阻塞；或者说，就是流水线思想在「计算—通信」上的翻版。

数学与手算：bucket 化

直接「每层梯度一算完就发」有个问题：一层一层的梯度是小包，每次集合通信都有固定的启动开销（建链、同步、协议头），小包一多，开销反而超过传输本身。解法是 **bucket 化**：设一个桶（约 25 MB），反向算出的梯度按序填桶，桶满即发，不等整层凑齐也不逐层单发。

手算。 7B 模型 bf16 梯度共 $7 \times 10^9 \times 2\ \mathrm{B} = 14\ \mathrm{GB}$：

$$\text{桶数} = \frac{14\ \mathrm{GB}}{25\ \mathrm{MB}} \approx 560\ \text{桶}$$

每桶在 50 GB/s 机间网络上的传输时间约 $\frac{25\ \mathrm{MB}}{50\ \mathrm{GB/s}} = 0.5\ \mathrm{ms}$，而反向传播一层通常要数毫秒，一层的时间足够藏住好几桶的传输。为什么桶取 25 MB 而不是更大或更小？太小了包太多、固定开销主导；太大了要等很久才能填满一桶，等待期间通信没法启动，重叠窗口被浪费。25 MB 是「固定开销可忽略、填充延迟可接受」的经验平衡点。

有一个边界要留意：反向最先算完的是**最后一层**，它的桶发得最早、藏得最好；**第一层**的梯度最后才算完，它的桶没有后续计算可以遮挡，这段「尾部通信」是暴露的。所以重叠能把通信成本压到接近零，但不是严格为零。

工程含义

重叠生效的前提条件是「反向计算时间 $>$ 通信时间」，即每层的算力密度足够。这就是为什么 TP 放机内（NVLink 900 GB/s 足够快不需要藏）、DP 的梯度同步走机间（必须靠重叠藏）的分工是合理的。PyTorch DDP、FSDP 默认都开启 bucket 化重叠，桶大小是可调参数；如果你看到 NCCL 时间和反向计算时间几乎完全重合的 profile，说明重叠工作正常，如果两者串行，优先检查 bucket 配置和梯度就绪顺序。

5.6 容错与 checkpoint：故障不是意外，是日程表

直觉与类比

单张 GPU 很可靠，但 1024 张卡组成的同步训练系统里，**任何一张卡挂掉，整个训练就停**（AllReduce 需要全员到齐）。这和 RAID 的经典论述一模一样：单盘年故障率不高，几百块盘组在一起，故障就从「万一」变成「例行」。大规模训练的第一公理：故障是常态，设计要围绕它展开。

手算：MTBF

设单卡年故障率约 5%（年化口径，含 GPU 本身及板卡、电源等关联硬件）。1024 卡集群的期望故障次数：

$$1024 \times \frac{0.05}{365} \approx 0.14\ \text{次/天}$$

也就是**大约每周一次**。万卡集群同样算法：

$$10^4 \times \frac{0.05}{365} \approx 1.4\ \text{次/天}$$

而且这只是硬件账。再计入网络抖动导致的 NCCL 超时、驱动崩溃、OOM、存储抖动等软件层故障，工程界的经验口径是**万卡集群每天发生数次故障**。也就是说，一个跑两个月的万卡训练，要经历上百次中断——不设计容错，这个项目在数学上就不可能完成。

checkpoint 频率的手算。 设 1024 卡集群约每 7 天出一次故障，每次恢复（重启、重载、重排队）约 10 分钟。若每 2 小时存一次 checkpoint，单次故障最坏丢 2 小时的训练进度，加上 10 分钟重启，损失约 2.2 小时；若每 12 小时存一次，最坏丢 12 小时。每次故障的期望损失约为「checkpoint 间隔的一半 + 重启时间」，间隔越短损失越小，但写入本身也耗时间——下一节会看到这直接体现在 MFU 的「故障重启」一项里。

checkpoint 的工程做法

异步写入。 朴素做法是所有卡同步阻塞写盘：训练停下，state dict 写 10 分钟，再继续。1024 卡每 2 小时停 10 分钟，等于白白损失约 8% 的算力。标准做法类似 Redis 的 bgsave：先在 CPU 内存里做一份快照（或 copy-on-write），训练立刻继续，后台线程慢慢把快照刷到磁盘。训练进程只付出一次内存拷贝的代价。

分层存储。 checkpoint 的写入路径分两层：先落**本地 NVMe**（速度快、几分钟内完成、不占训练时间），再异步搬运到**远端对象存储**做持久化（本地盘会随节点故障一起丢）。保留策略通常是本地留最近 2~3 份供快速回滚（配合 5.3 的 spike playbook），远端按小时/按天留周期快照。还要注意写入风暴：1024 卡同时写 TB 级文件会打爆存储集群，需要聚合写入或分片错峰，第 8 章会展开。

弹性训练。 进一步的做法是节点挂掉后不再全员停下等人修，而是自动把坏节点踢出、缩小数据并行组继续训练，修好后再加回来——把「停机」降级成「降速」。

5.7 MFU：这 1024 张卡到底有几张在干活

直觉与类比

买卡按峰值算力付钱，干活按实际吞吐结算。**MFU（Model FLOPs Utilization）**就是两者的比值：

$$\text{MFU} = \frac{\text{实测吞吐（FLOPS）}}{\text{峰值算力（FLOPS）}} = \frac{T_{\text{理想}}}{T_{\text{实际}}}$$

它就是训练集群的「资源利用率 SLO」，和你在意 CPU 利用率、广告填充率是同一心态：钱花出去了，有多少变成了有效产出。

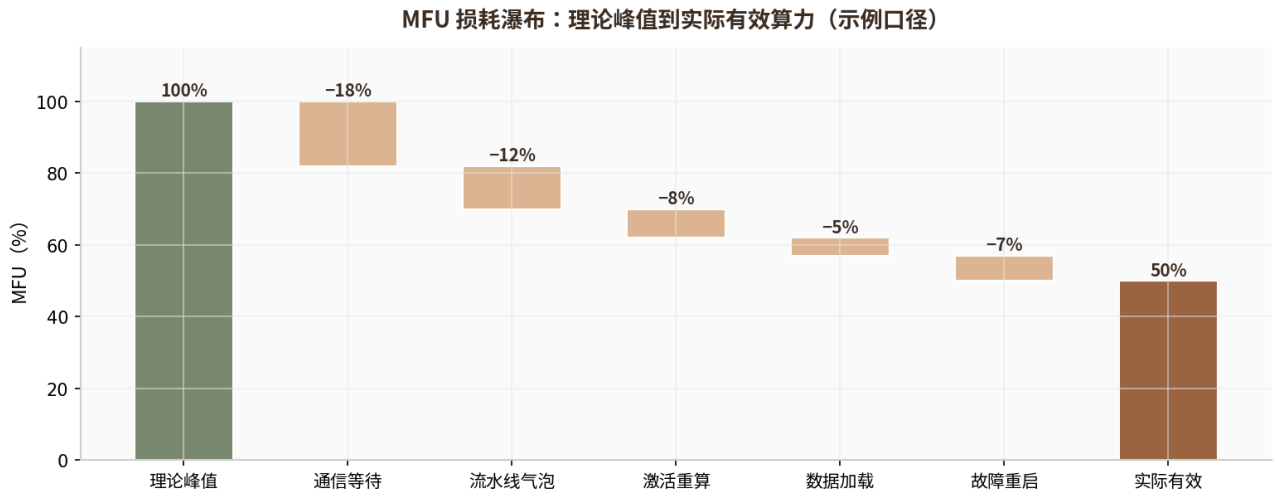
手算：A5 续算

回到附录 A5 的账：7B 模型、2T tokens，总计算量 8.4×10^{22} FLOPs；1024 卡，单卡 bf16 峰值约 10^{15} FLOPS，集群峰值约 1.0×10^{18} FLOPS。理想工期（A5 的计划值）约 1.9 天。

假设实测跑了 **3.8 天**才训完：

$$\text{MFU} = \frac{T_{\text{理想}}}{T_{\text{实际}}} = \frac{1.9}{3.8} = 50\%$$

含义很直白：一半的卡时在做有效计算，另一半被各种损耗吃掉了。（口径说明：这里把 A5 的 1.9 天当作理想工期；若严格以硬件一秒都不浪费的纯峰值下界 $T_{\min} = \frac{8.4\times10^{22}}{1.0\times10^{18}} \approx 8.4 \times 10^{4}$ 秒 ≈ 0.97 天为基准，3.8 天对应约 25%，即 A5 的计划本身就按 MFU 50% 打过折。读业界报告时务必先问对方用哪个口径。）



损耗从哪来：一张瀑布账

从 100% 峰值到 50%，中间消失的部分大致这样分布：

损耗来源	典型占比	对应章节
通信等待（AllReduce 没藏住的部分、尾部通信）	约 15%	4.2、5.5
流水线 bubble（PP 启动与排空）	约 10%	4.4
数据加载等待（DataLoader 跟不上 GPU）	约 5%	3.6
梯度检查点的重算开销	约 10%	5.4
故障重启与 checkpoint 回滚	约 10%	5.3、5.6
剩余有效计算	**约 50%**	—

这张表就是本章的合订本：每一项损耗都对应前面某一节的技术，每一项技术都是把瀑布里的某一格往回抢。业界大模型预训练的 MFU 通常在 30% 到 50% 之间，能稳定跑到 50% 以上已经是第一梯队。

工程含义

MFU 是训练团队的终极 KPI，因为它直接换算成钱：MFU 从 40% 提到 50%，等价于同样的预算多出 25% 的算力，或者说同一个模型早 20% 的时间交付。第 8 章做成本核算时会看到，每百万 token 的训练成本就是「总 FLOPs ÷ MFU ÷ 卡时单价」的函数——MFU 是公式里唯一一个靠工程质量决定、而不是靠采购决定的变量。这也是 AI Infra 工程师价值的最终度量。

第 6 章 推理原理：Prefill、Decode 与 KV Cache

第 2 到第 5 章都在讲训练：一个目标函数、一条 loss 曲线、一次覆盖整个 batch 的参数更新。从这一章开始换战场——推理。推理没有反向传播、没有 optimizer、没有跨卡同步梯度，看起来「只是前向一遍」，工程难度却一点不低：训练的优化目标是「单一大任务的吞吐」，推理的优化目标是「海量并发小请求的延迟与吞吐的平衡」，这和分布式后台工程师熟悉的在线服务问题同构。本章先把推理的计算性质讲透：为什么一次推理必然分成 Prefill 和 Decode 两个阶段，为什么前者卡在算力、后者卡在显存带宽，以及 KV cache 这本显存账怎么算。理解了这三件事，第 7 章的 vLLM、连续批处理、PD 分离就只是水到渠成的工程解法。

6.1 推理 vs 训练：一次只能走一步

直觉。训练像批处理作业：一次性喂入海量数据，并行处理，跑完收工。推理像在线服务：请求随时到达，逐个处理，且每个请求的处理过程本身是一环扣一环的串行链条。

为什么串行。大语言模型的生成方式是自回归（auto-regressive）：模型每次前向只输出下一个 token 的概率分布，采样得到一个 token，把它拼回输入，再跑下一次前向。第二册讲过，因果掩码保证了位置 t 只能看到位置 1 到 $t-1$ 的内容——这既是训练时的约束，也决定了推理时的结构：第 t 个 token 的生成依赖前 $t-1$ 个 token 的全部计算结果，无法提前并行。想生成 200 个 token，就必须串行执行 200 次前向，没有捷径。

这里有一个容易混淆的点：训练时同样用因果掩码，却能并行。区别在于训练时所有位置的「正确答案」已知（就是语料里的下一个词），200 个位置可以排成矩阵同时算损失；推理时第 t 个位置的输入依赖第 $t-1$ 步的**输出**，答案还没生成出来，无从排矩阵。一句话：训练是「答案已知的并行填空」，推理是「边写边决定下一句的串行接龙」。

这是推理与训练最本质的区别，用后台语言对照一下：

维度	训练	推理
任务模式	单一大 batch 作业，跑数天	海量并发请求，每个几十毫秒到几十秒
并行粒度	数据/张量/流水线三维并行	只能并行「不同请求」，不能并行「同一请求的不同 token」
核心指标	MFU、总时长	TTFT、TPOT、P99、goodput
瓶颈	算力与集合通信带宽	显存容量与显存带宽
类比	Hadoop 批作业	高并发 RPC 服务

工程含义。推理引擎本质上是一个有状态的高并发在线服务：每个请求带着不断增长的上下文（KV cache），生成过程不可并行、不可抢占后无损恢复（除非保留 KV），且延迟指标按 token 粒

度计费。你在后台系统里学到的限流、排队、调度、缓存、容量规划，全部用得上，只是「资源」从 CPU 和连接数变成了显存和显存带宽。

还有一个与直觉相反的结论值得记住：**推理的总 FLOPs 远小于训练**（一次推理约 $2N\times$ 总 token 数，对比第 2 章训练的 $6ND$），但推理的钱并不比训练省。原因在于推理的瓶颈不是算力而是带宽与显存，GPU 的乘加单元大量空转，MFU 远低于训练——第 8 章做成本核算时会用到这一点。

6.2 Prefill 与 Decode：两种完全不同的负载

一次推理请求的生命周期严格分成两个阶段：

- **Prefill（预填充）**：收到长度为 L 的 prompt 后，一次性并行处理这 L 个 token，算出第一个输出 token，顺带把整段 prompt 的 KV cache 建好（每个位置在每一层的 K、V 都落进显存，供 Decode 阶段反复读取）。
- **Decode（解码）**：从第二个 token 开始，每步只处理 1 个新 token，串行循环，直到生成结束符或达到长度上限。每步产生 1 个新 token 的同时，也向 KV cache 追加 128 KB（A4 口径）。

这两个阶段跑的是同一个模型、同一组权重，但计算性质截然相反，优化手段也因此分道扬镳。用后台的话讲：Prefill 像「一次性的大 Map 任务」，可以宽并行吃满算力；Decode 像「高频小请求的死循环」，每一步都要全量读一遍状态。把两者拆开来认识，是理解一切推理优化技术的前提。

手算：Prefill 是计算密集

沿用附录 A 的 7B 模型。回忆第 2 章的结论：一次前向的 FLOPs 约等于 $2N$（N 为参数量），即每个 token 流过模型约做 $2\times 7\times 10^9 = 1.4\times 10^{10}$ 次乘加。Prefill 把 1000 个 token 的 prompt 当作一个大矩阵乘并行处理（忽略随长度平方增长的注意力项，它在 $L=1000$ 时占比很小）：

$$\text{FLOPs}_{\text{prefill}} \approx 2N \times 1000 = 2\times 7\times 10^9 \times 10^3 = 1.4\times 10^{13}$$

按附录 A5 的单卡有效算力 5×10^{14} FLOPS 计：

$$T_{\text{prefill}} = \frac{1.4\times 10^{13}}{5\times 10^{14}} = 2.8\times 10^{-2}\ \text{s} = 28\ \text{ms}$$

1000 token 的 prompt 只需约 28 ms，因为 L 个 token 拼成大矩阵后，每次从显存读出的权重都被复用了 L 次，GPU 的乘加单元能跑满。

两个补充说明。其一，这里省略了注意力项：注意力的计算量随 L^2 增长，$L=1000$ 时相对 $2NL$ 的占比约百分之几，可以忽略；当 prompt 达到数万 token 时它才会成为主角，届时需要第 7 章 FlashAttention 之类的技术。其二，Prefill 的最后一步还要做采样：对词表（约 10 万项）做一次矩阵乘加 softmax 得到第一个 token 的分布，这一步开销很小，计入 Prefill。

手算：Decode 是访存密集

Decode 每步只来 1 个新 token，FLOPs 同样约是 $2N$：

$$\text{FLOPs}_{\text{decode}} \approx 2N = 1.4 \times 10^{10}$$

算力角度只需 $1.4 \times 10^{10} / 5 \times 10^{14} = 2.8 \times 10^{-5}$ s，即 0.028 ms。但问题是：处理这 1 个 token 依然要把全部 14 GB 权重（附录 A2，bf16）从 HBM 读一遍。按 HBM 带宽 3 TB/s：

$$T_{\text{decode}} \geq \frac{14\ \text{GB}}{3000\ \text{GB/s}} \approx 4.7 \times 10^{-3}\ \text{s} = 4.7\ \text{ms/token}$$

也就是说，Decode 一步的实际耗时约 4.7 ms，是计算所需时间的约 170 倍——GPU 的乘加单元 99% 以上时间在等数据。单请求 Decode 的理论吞吐上限约为 $1000/4.7 \approx 213$ token/s，与 batch 多大、算力多强都无关，只取决于显存带宽（还没计入读取 KV cache 的开销，见 6.3）。

这正是「矩阵-向量乘」与「矩阵-矩阵乘」的差别：Decode 每步的权重矩阵只与 1 个 token 的向量相乘，每个权重元素读完只参与一次乘加，权重读带宽成为唯一瓶颈。想提升单请求的 Decode 速度，路子只有三条：用更高带宽的硬件（HBM 代际升级）、让权重更小（量化，第 7 章）、或者一次读出权重后「假装」算了多个 token（投机解码，第 7 章）。算力升级对 Decode 毫无帮助——这和「IO 密集服务加 CPU 核数没用」是同一个道理。

算术强度：一把量出瓶颈的尺子

这个反差用**算术强度**（arithmetic intensity，每读 1 字节数据做多少次运算）可以一句话说清：

$$\text{算术强度} = \frac{\text{FLOPs}}{\text{访存字节数}}$$

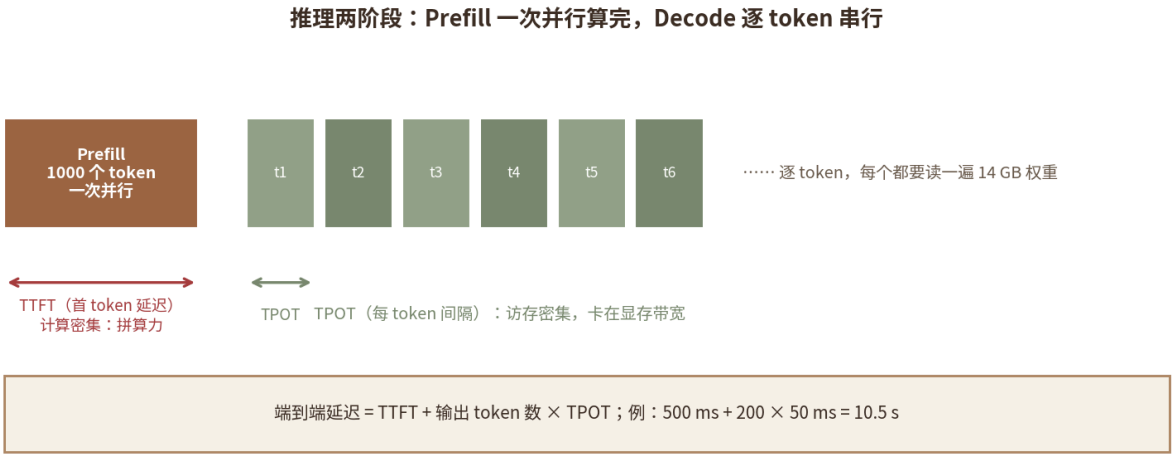
- Prefill（1000 token）：$1.4 \times 10^{13}\ /\ 1.4 \times 10^{10}\ \text{B} = 1000$ FLOP/B
- Decode（1 token）：$1.4 \times 10^{10}\ /\ 1.4 \times 10^{10}\ \text{B} = 1$ FLOP/B

H100 级单卡的「盈亏平衡点」约为峰值算力除以显存带宽：$10^{15}\ /\ 3 \times 10^{12} \approx 333$ FLOP/B。算术强度高于 333 是计算密集（卡在算力），低于 333 是访存密集（卡在带宽）。Prefill 的 1000 远在平衡点之上，Decode 的 1 远在平衡点之下。

阶段	计算量	访存量	算术强度	瓶颈	对应后台负载
Prefill	$2NL$	权重读 1 遍	$\approx L$ FLOP/B	算力	CPU 密集批处理
Decode	$2N$ / token	权重 + KV 读 1 遍	≈ 1 FLOP/B	显存带宽	IO 密集在线服务

工程含义。这张图的两条推论会贯穿第 7、8 章：其一，Decode 阶段把多个请求拼成 batch，权重只读一遍就能服务 B 个 token，算术强度近似乘 B，因此「攒 batch」是推理引擎提升吞吐的第一杠杆——batch 从 1 加到 32，单步耗时只从约 4.7 ms 涨到十几毫秒（多读了 32 份 KV cache），吞吐

却翻了约 30 倍；其二，Prefill 吃算力、Decode 吃带宽，两类负载混在同一池子里互相干扰：一个 10 万 token 的长文 Prefill 会独占 GPU 几百毫秒，期间所有 Decode 请求的 TPOT 全部抖动，这正是第 7 章 chunked prefill 与 PD 分离要解决的调度问题。



6.3 KV Cache：用显存换重复计算

为什么需要缓存。Decode 第 t 步时，注意力需要拿新 token 的 query 去和前 $t-1$ 个 token 的 key、value 做点积（第二册已推导注意力公式，此处不重复）。如果不缓存，每一步都要把前 $t-1$ 个 token 的 K、V 从头重算一遍——第 t 步的计算量随 t 线性增长，生成 L 个 token 的总开销变成平方级。KV cache 的做法很直白：每个 token 第一次算出的 K、V 全部留在显存里，后续步骤直接读。这是典型的「空间换时间」，和你给热点数据加 Redis 缓存是同一个思想，只不过这里的「缓存项」必须驻留在显存里，因为每步都要被 GPU 读一遍。

逐项推导占用量（附录 A4，7B：32 层、GQA 8 个 KV 头、head_dim $=128$、bf16 每元素 2 B）。每个 token 在每层要存 K 和 V 两份向量：

$$2\ (\text{K 和 V}) \times 8\ (\text{KV 头}) \times 128\ (\text{head_dim}) \times 2\ \text{B} = 4096\ \text{B} = 4\ \text{KB}/\text{层}/\text{token}$$

乘以 32 层：

$$4\ \text{KB} \times 32 = 128\ \text{KB}/\text{token}$$

4096 token 上下文的单条序列：

$$128\ \text{KB} \times 4096 = 512\ \text{MB}$$

这笔账还回答了「为什么不重算」：重算一个 token 的 K、V 需要让它再过一遍模型（$2N$ FLOPs 且读一遍权重），而读它缓存的 K、V 只要 128 KB 显存带宽——在 3 TB/s 下约 0.04 微秒。缓存把「每步 $O(N)$ 的重算」变成「每步 $O(1)$ 的读取」，代价只是显存。对访存密集的 Decode 来说这是毫无疑问的划算买卖。

注意 GQA 的作用已经体现：若是传统 MHA（32 个 KV 头），这里会是 $512\ \mathrm{KB/token}$、单序列 2 GB——把 KV 头从 32 砍到 8，KV cache 直接除以 4，几乎不掉效果，这是模型结构层面为推理省显存的标准做法。对照表如下：

结构	KV 头数	每 token 全模型	4096 上下文单序列
MHA	32	512 KB	2 GB
GQA（本算例）	8	128 KB	512 MB
MQA 极限	1	16 KB	64 MB

另外注意推导里的三个因子各有含义：乘 2（K 和 V）不可省；层数与 head_dim 由模型结构定死；唯一能由工程侧调整的只有**精度**——KV cache 从 bf16 量化到 int8 直接减半，代价是轻微精度损失，这是线上大模型服务的常用手段。

线性关系与批量放大。KV cache 与「序列数 × 上下文长度」严格线性。batch 32、每条 4k 上下文：

$$32 \times 512\ \mathrm{MB} = 16\ \mathrm{GB}$$

此时 KV cache（16 GB）已经超过了权重本身（14 GB）。一张 80 GB 的卡，装 7B 模型推理，权重占 14 GB，剩下的显存大头几乎全是 KV cache；它直接决定了能并发多少请求、能支持多长上下文。反过来说容量规划：80 GB 卡扣掉权重 14 GB、框架与碎片预留约 10 GB，剩约 56 GB 给 KV cache，按 512 MB/序列只能容纳约 $56\ \mathrm{GB}/512\ \mathrm{MB} \approx 112$ 条 4k 序列并发——这就是这张卡并发上限的硬顶，第 7 章所有调度优化都在逼近这个上限。

再对照 Decode 的时间账：上下文 4096 时，每步还要额外读 512 MB KV cache，约多 $512/3000 \approx 0.17$ ms——上下文越长，Decode 越慢，这也是线性恶化的。超长上下文（128k 级）场景中 KV cache 的读取时间甚至会超过权重读取，成为 Decode 的第一瓶颈。

工程含义。KV cache 是推理系统里的「一等资源」，地位相当于后台服务的内存池：它的总量是硬约束，它的分配效率决定并发上限。第 7 章的 PagedAttention（借鉴 OS 虚拟内存的分页管理）、前缀缓存、KV 量化，全都是围绕这本账展开的。

6.4 服务质量指标：TTFT、TPOT 与 Goodput

后台服务用 P99 延迟和 QPS 描述服务质量；推理服务因为「首 token 之后才进入流式输出」，需要把延迟拆成两段：

- **TTFT**（Time To First Token）：请求到达到第一个输出 token 返回的耗时，约等于排队时间 + Prefill 时间。它决定用户「等多久看到第一个字」。
- **TPOT**（Time Per Output Token）：Decode 阶段相邻两个 token 的平均间隔。它决定流式输出「刷字快不快」。

- **端到端延迟**：TTFT + 生成 token 数 × TPOT。
- **吞吐**：单位时间系统产出的总 token 数（token/s），是容量规划和成本核算的口径。

SLO 分解手算

假设产品 SLO 为 TTFT ≤ 500 ms、TPOT ≤ 50 ms，一个典型请求生成 200 token：

$$T_{\text{端到端}} = 500\ \text{ms} + 200 \times 50\ \text{ms} = 10500\ \text{ms} = 10.5\ \text{s}$$

对照 6.2 的手算：1000 token 的 Prefill 理论值约 28 ms，单请求 Decode 理论值约 4.7 ms/token，似乎绰绰有余。但真实系统里 TTFT 的大头是**排队**（前面的请求占着 Decode 的显存槽位，新请求的 Prefill 进不来），TPOT 的大头是**大 batch 下 KV cache 读取量的叠加**：batch 内 32 条序列各带 512 MB KV cache，每步合计要读 16 GB，仅 KV 读取就超过 5 ms，比权重读取还慢。这就是为什么推理 SLO 不能拿理论值做容量规划，必须用压测的 P99 数据——和后台服务「均值没意义，看分位数」的经验完全一致。

还有一个口径换算值得练习，因为业务方只会给你 QPS：若平均每请求生成 200 token，1 QPS 意味着 Decode 吞吐 200 token/s。单卡理论上限 213 token/s（6.2 节），即单卡撑不住 1 QPS——必须靠 batch 把权重读取摊薄到 B 个请求头上，这正是吞吐与延迟此消彼长的根源。

与 P99/QPS 的对应关系

后台指标	推理对应物	说明
首字节延迟 TTFB	TTFT	都反映排队 + 首次计算
流式响应速率	TPOT 的倒数	TPOT 50 ms ≈ 每秒刷 20 字
P99 延迟	TTFT/TPOT 的 P99 分位	批处理挤占使长尾更严重
QPS	请求/s 与 token/s 两个口径	token/s 才是算力消耗的真实口径
SLA/SLO	TTFT、TPOT 双阈值	两个指标要分别达标

Goodput。把 QPS 打高不难，难的是打高之后 SLO 还在。于是推理领域引入 **goodput（有效吞吐）**：单位时间内**满足 SLO**（TTFT 和 TPOT 同时达标）的请求数。超出 SLO 的请求虽然在产出 token，但用户体验已经不合格，不计入 goodput。这和你给后台服务定「P99 < 200 ms 前提下的最大 QPS」是同一个概念。goodput 是衡量推理引擎和调度策略优劣的统一标尺：连续批处理、PD 分离等技术比的就是同一硬件上谁的 goodput 高。

一个直觉化的图景：随着负载升高，原始吞吐（token/s）先线性后趋于饱和，而 goodput 会在某个临界点之后**掉头向下**——队列堆积让 TTFT 大面积超时，新挤进来的请求反而拖垮已有请求。这和你见过的线程池/连接池过载崩溃曲线一模一样，因此推理网关侧的限流阈值必须按 goodput 拐点而不是按吞吐饱和点来设定。

把这个账接回 6.3 的容量上限做一个完整手算：单卡 KV cache 上限约 112 条并发序列，若 SLO 要求 TPOT ≤ 50 ms，而 batch 112 时每步合计要读 $112 \times 512\ \text{MB} \approx 57\ \text{GB}$ KV cache，仅这一项就需 $57000/3000 \approx 19$ ms，加权重 4.7 ms 后仍在 50 ms 以内——此时 TPOT 达标，goodput 瓶颈转移到 TTFT 的排队上。一旦并发再翻倍（228 条），KV 读取单步就要约 39 ms，TPOT 逼近 SLO 红线，goodput 开始崩。这个「先撞 TTFT、再撞 TPOT」的顺序，是推理容量规划的基本节奏。

6.5 采样参数的工程面

模型每步输出的不是 token，而是词表上的概率分布（logits 经 softmax，第二册已推导）。采样参数决定怎么从这个分布里挑一个 token，这属于「算法面」；但参数的选择会实打实地改变缓存命中率、显存占用和容量规划，这才是工程读者要关心的部分。数学上各一句话：

- **temperature** T：把 logits 除以 T 再做 softmax。$T > 1$ 分布变平（更随机），$T < 1$ 分布变尖（更确定），$T \to 0$ 退化为 argmax（贪心，总取概率最大者）。
- **top-p**（nucleus sampling）：只从累计概率达到 p 的最小 token 集合里采样，把长尾直接截掉。

对原理推导本章到此为止；但采样参数有几个**纯工程**的影响，经常被忽视：

其一，temperature 决定可缓存性。$T = 0$ 的贪心解码是确定性的：相同前缀必然产生相同输出，因此前缀缓存（多个请求共享 system prompt 的 KV cache）甚至整段响应缓存都能无损命中。$T > 0$ 引入随机性，同样输入可能产出不同续写，前缀之后的缓存价值大打折扣。业务侧如果对确定性有要求（如代码生成、结构化抽取），$T = 0$ 不只是质量问题，还是成本问题——一个全员 $T = 0$ 的业务，公共前缀的 Prefill 只需算一次，TTFT 与算力成本双双下降。

其二，采样不影响 KV 复用与批处理结构。采样只发生在每步最后「从分布挑 token」的轻量环节，完全不碰前面的 Transformer 计算和 KV cache。不同 temperature、top-p 的请求可以混在同一个 batch 里共享权重读取；KV cache 的分配与采样参数无关。这意味着采样参数的差异化是「免费」的——不像 max_tokens 那样直接改变显存占用时长。

其三，max_tokens 是资源参数，不是质量参数。它决定一条请求占用 KV cache 槽位的时长上限。容量规划时，平均输出长度直接乘进 6.3 的显存账和 6.4 的延迟账；运营侧乱调 max_tokens，效果等同于后台服务里把超时时间随手翻倍——长尾请求会占住稀缺资源，挤占队列里所有请求的 TTFT。

其四，流式输出（SSE）不是锦上添花，而是指标的一部分。TPOT 的定义依赖逐 token 推送；若把响应攒到最后一次性返回，用户感知的「首字节延迟」就变成了端到端延迟 10.5 s，TTFT 指标形同虚设。工程上这意味着推理网关必须全链路支持流式：负载均衡不能缓冲响应、客户端要处理断流重试，这些与 SSE/WebSocket 长连接服务的运维经验完全相通。

小结。本章完成了推理侧的「原理建模」：自回归决定串行结构（6.1），算术强度区分出计算密集的 Prefill 与访存密集的 Decode（6.2），KV cache 把显存变成一等资源且随序列数与长度线性膨胀

（6.3），TTFT/TPOT/goodput 给出服务质量口径（6.4），采样参数则提醒我们确定性本身是一种可缓存、可省钱的资源（6.5）。第 7 章将在此之上回答工程问题：如何用连续批处理、PagedAttention、量化、投机解码与 PD 分离，把 goodput 压到硬件极限。

第 7 章 推理工程：从 vLLM 到 PD 分离

第 6 章我们回答了「推理是什么」：Prefill 并行吃算力，Decode 串行吃显存带宽，KV cache 随上下文线性膨胀。这一章回答「推理怎么做成一个高吞吐、低延迟、可扩缩容的在线服务」。这是你作为分布式后台工程师最熟悉的战场——调度、内存管理、缓存、负载均衡、读写分离——只不过每一项都因为 GPU 和自回归生成的特性，长出了一副新面孔。

我们将沿着 vLLM 这条主线走：先看调度层的连续批处理，再看显存层的 PagedAttention，然后是模型压缩层的量化、算子层的 FlashAttention、算法层的投机解码，最后上升到集群层的 PD 分离，并用一个请求的完整旅程收束全章。

7.1 连续批处理：别让最慢的请求拖住整个 batch

直觉。 第 6 章说过，Decode 每步只产出 1 个 token，GPU 利用率极低，唯一的出路是把很多请求凑成 batch 一起跑。问题是：怎么凑？

最朴素的办法是**静态批处理**（static batching）：攒够 B 个请求，一起开跑，等**所有**请求都生成完毕，再放下一批进来。这就像你熟悉的「一次性 barrier」——一批线程全部到达屏障后才能继续。问题显而易见：同一个 batch 里，有的请求生成 50 个 token 就结束了，有的要生成 2000 个。短请求结束后，它占的 slot 就空转，GPU 算力被尾巴上最长的那个序列白白拖住。

类比。 静态批处理是「一次性 barrier」，**连续批处理**（continuous batching，也叫 iteration 级调度）是「线程池」：有任务完成就立刻释放 worker，队列里有新任务就立刻补进来，worker 永不空转。

机制。 连续批处理把调度粒度从「请求级」细化到「iteration 级」。每跑完一步 Decode（所有在跑序列各产出 1 个 token），调度器检查一遍：

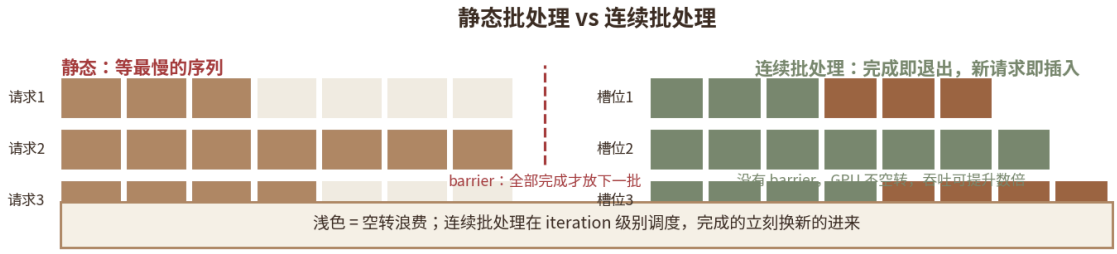
- 谁的输出命中了 EOS 或达到长度上限？立刻把它从 batch 里移除，释放 KV cache；
- 等待队列里有新请求吗？只要显存够，立刻在**下一步** iteration 插入（这一步通常要先跑它的 Prefill，即 prompt 计算，业界也称 chunked prefill 混合调度）；
- 显存不够插不进去？继续排队，或者在极端情况下把某些请求抢占（preempt）出去。

于是 batch 的组成是流动的：每一轮 iteration 里，完成的退出、新来的插入，GPU 始终跑满。

手算例子。 假设有 4 个请求，需要生成的长度分别是 10、10、10、100 个 token。静态批处理下，batch 必须跑满 100 步才放下一批，前 3 个请求在后 90 步里空占 slot，有效产出是 $10+10+10+100=130$ 个 token，但消耗了 $4\times 100=400$ 个「slot·步」，利用率 $130/400=32.5\%$。连续批处理下，第 10 步结束时 3 个短请求立刻退出，空出的 3 个 slot 马上可以接新请求；如果队列里请求源源不断，从第 11 步起这 3 个 slot 就一直在为新请求产出 token，slot 利用率逼近 100%。实际系统中，连续批处理对吞吐的提升通常在数倍量级（vLLM 论文报告相对朴素的离线

批处理有数量级改善，相对仅做连续批处理的系统也有 2–4 倍，后者主要来自下一节的 PagedAttention）。

工程含义。 连续批处理改变了你对「请求」的建模：推理服务不再是「一次 RPC 一个来回」，而是一个常驻的调度循环，请求是可以在任意 iteration 边界进出调度器的。这带来两个直接后果：第一，**TTFT 与 TPOT 解耦**——新请求插队会稍微拖慢在跑请求的 TPOT，调度策略（等待队列长度、最大 batch、chunked prefill 的块大小）就是在首 token 延迟和稳态吞吐之间调参；第二，**抢占与重算**成为标配——当 KV cache 不够时，vLLM 会把低优先级请求的 KV 全部丢弃（recompute 模式）或换出到 CPU 内存（swap 模式），等显存宽裕再恢复，这和你熟悉的「内存不足时的换页」完全是同一套思路。



7.2 PagedAttention：给 KV cache 做虚拟内存

直觉。 上一节说「显存够就插入」，但「够不够」远比听起来麻烦。第 6 章 A4 算过：7B 模型每 token 的 KV cache 是 128 KB，4096 token 上下文单序列约 512 MB。传统实现为每个请求**按最大可能长度预留连续显存**——比如一律按 4096 token 预留 512 MB。这有两个浪费：

- **预留浪费**：实际只生成 200 个 token 的请求，也占着 512 MB；
- **碎片浪费**：请求进出导致显存被切成大小不一的空洞，就像 malloc/free 久了的堆，总空闲量够、连续块不够。

vLLM 论文的实测是：这种预留式管理下，KV cache 显存的真正利用率只有 20%–40%，超过一半都浪费在预留和碎片上。

类比。 这正是操作系统在几十年前解决的问题。进程要「连续地址空间」，物理内存是碎片化的，怎么办？虚拟内存分页：把地址空间切成固定大小的页，用页表把虚拟页映射到物理页，物理页按需分配、可以散落各处。**PagedAttention 就是把这套机制原样搬到 KV cache 上**：

操作系统	PagedAttention
虚拟地址空间	请求的逻辑 token 序列
页（page，如 4 KB）	KV 块（block，如 16 token）
页表	block table（每个请求一张）
物理页帧	显存中的物理 KV 块
按需分配/换出	生成到新块才分配；抢占时换出到 CPU 内存
fork 的 copy-on-write	并行采样/beam search 共享前缀块，写时复制

机制。 以块大小 16 token 为例：一个请求的 KV cache 在逻辑上是连续的 token 序列，物理上则被切成若干 16 token 的块，散落在显存的块池里，请求的 block table 记录「第 i 个逻辑块 $\rightarrow$ 第 j 个物理块」的映射。生成新 token 时，只有当前块写满了才申请新块；注意力计算时，kernel 按 block table 去取各个物理块。因为块大小固定，**外部碎片直接归零**——任何空闲块都能给任何请求用；浪费只剩下每个请求最后一个块没写满的部分，平均半个块，即 8 个 token 的 KV（约 1 MB），相对 512 MB 的预留浪费可以忽略。

手算例子。 一台 80 GB 显存的 H100，7B 模型权重占 14 GB（A2），留给 KV cache 约 60 GB，即 $60 \times 1024/0.125 \approx 491520$ 个 token 的容量（每 token 128 KB $= 0.125$ MB）。预留式管理下，若一律按 4096 token 预留，只能同时容纳 $491520/4096 = 120$ 个请求，而真实平均上下文若是 512 token，实际只用到 $120 \times 512 = 61440$ 个 token 的 KV，利用率 12.5%。分页式管理下，按需分配，491520 个 token 的容量除以真实平均 512 token，可同时容纳 960 个请求——batch 上限直接放大 8 倍。batch 大了，Decode 每步读一遍 14 GB 权重所摊到的 token 数就多了 8 倍，吞吐近似同比例提升。这就是 vLLM 论文中 2–4 倍吞吐提升的主要来源。

工程含义。 第一，PagedAttention 是连续批处理的**地基**——没有按需分配，7.1 的「显存够就插入」根本无法高效判断和执行；第二，固定块大小让「前缀共享」变得免费：同一 system prompt 的多个请求可以共享前缀的物理块（reference counting），这对有大量公共前缀的业务（多轮对话、few-shot 模板）是又一笔显存红利，vLLM 的 automatic prefix caching 即基于此；第三，块大小是 trade-off：块越小碎片越少，但 block table 越长、kernel 寻址开销越大，16 是经验上的甜点。

7.3 量化：用整数近似浮点，显存直接减半

直觉。 第 6 章说过 Decode 卡在显存带宽：每生成 1 个 token 就要把 14 GB 权重读一遍。既然瓶颈是「读」，最直接的优化就是让权重变小——把 bf16 的 2 字节权重压成 1 字节（INT8）甚至半个字节（INT4）。这就是**权重量化**。

类比。 这和你熟悉的压缩是一个思路：用更短的编码表示数据，传输（这里是「从 HBM 读到 SM」）更快，代价是有损。PNG 是无损的，JPEG 是有损的；量化就是有损压缩，压缩率越高，画质（精度）损失越大。

数学。 最简单的**对称量化**（symmetric quantization）把一个浮点数组映射到 int8 的 $[-127, 127]$ 区间。先求缩放因子（scale）：

$$s = \frac{\max |x|}{127}$$

然后每个元素做舍入：

$$x_{int} = \text{round}\left(\frac{x}{s}\right)$$

反量化（还原成浮点参与计算）：

$$x_{dq} = x_{int} \times s$$

「对称」指正负区间关于 0 对称，不需要额外的零点偏移，硬件实现最简单。

手算例子。 某组权重的范围是 $[-3.2, 3.2]$，即 $\max |x| = 3.2$。第一步算 scale：

$$s = \frac{3.2}{127} \approx 0.0252$$

第二步量化。取 $x = 1.0$：

$$x_{int} = \text{round}\left(\frac{1.0}{0.0252}\right) = \text{round}(39.68) = 40$$

存下来的就是一个字节的整数 40 和一份（每组共享的）scale。第三步反量化：

$$x_{dq} = 40 \times 0.0252 = 1.008$$

误差 $|1.008 - 1.0| = 0.008$，恰好在半个 scale（0.0126）以内——这是舍入量化的误差上界。注意 scale 可以按整层一组（per-tensor）也可以按输出通道一组（per-channel），粒度越细误差越小，工程上权重量化普遍用 per-channel。

取舍：W8A16 与 W4A16。 记号 WaAb 表示权重 a bit、激活 b bit：

方案	权重大小（7B）	相对 bf16	精度损失	适用场景
bf16	14 GB	$1\times$	无	显存充裕、追求最高质量
W8A16	7 GB	$1/2$	几乎无感（困惑度变化在小数点后）	主流线上选择
W4A16	3.5 GB	$1/4$	可测但可控（GPTQ/AWQ 等算法补偿）	单卡装大模型、边缘部署

两个要点：第一，激活保持 16 bit（A16），因为激活是「流经」模型的中间量，量化误差会逐层累积，对激活下手的风险远高于对权重下手；第二，既然计算仍在 16 bit 下进行（反量化后做矩阵乘），量化的收益主要是**显存占用减半/再减半、读带宽减半/再减半**——对访存密集的 Decode 来说，读带宽减半就近似等于 TPOT 减半。更激进的 W8A8（激活也量化）能用整数矩阵乘直接加速计算，但校准和精度成本高得多，本节点到为止。

工程含义。 量化是推理侧性价比最高的一刀：一行配置，显存减半，吞吐上去，SLO 几乎不动。代价是困惑度（perplexity，第二册讲过，可粗略理解为模型对文本的「惊讶程度」）轻微上升，量化位数越低上升越明显；工程上够用就行——先用 W8A16 上线，显存实在不够再上 W4A16。

7.4 FlashAttention：别让注意力矩阵在 HBM 里过夜

直觉。 量化解决「权重读得多」，FlashAttention 解决「中间结果读写多」。第二册推导过注意力：QK^T 得到一个 $L \times L$ 的分数矩阵，过 softmax，再乘 V。朴素实现会把这个 $L \times L$ 矩阵**物化**（materialize）到 HBM：写一次 QK^T、读出来算 softmax、再写回、再读出来乘 V。当 L 上到几千几万，这个矩阵的读写就是天文数字。

类比。 GPU 的存储层次和 CPU 完全同构：

CPU 世界	GPU 世界	容量（A100/H100 量级）	带宽
L1/L2 cache	SRAM（片上）	每卡合计约 20 MB 量级	约 19 TB/s
主存 DDR	HBM（显存）	80 GB	约 2–3 TB/s

快的小、慢的大，差近一个数量级。高性能编程的第一性原理在两边的写法也一样：**尽量让数据留在快的那一层，算完再写回**。FlashAttention 本质上就是一次针对注意力算子的「cache 优化」。

机制（只讲 IO 直觉，不重复第二册的公式）。 FlashAttention 做两件事：

- **分块（tiling）**：把 Q、K、V 都切成小块，每次只把几个小块从 HBM 搬进 SRAM，在片上算完这一小块对最终输出的贡献，再搬下一块。$L \times L$ 的完整矩阵从头到尾不落地。
- **在线 softmax**：难点在于 softmax 需要对整行求和做归一，分块后「整行」还没看完怎么办？答案是边看边维护两个滚动量——目前见过的最大值和归一化分母，每来一个新块就修正一次已累计的结果（这就是「online softmax」），全部看完后结果与一次性算完严格相等。反向传播时所需的 softmax 中间值不落盘，用时**重算**——用 SRAM 里的重计算换 HBM 的读写，与第 5 章梯度检查点「重算换显存」是同一思想。

IO 复杂度账。 朴素实现的 HBM 访问量随 L^2 增长（因为 $L \times L$ 矩阵要反复读写），FlashAttention 通过分块把 HBM 访问量降到近似随 L 线性（精确地说，注意力计算量本身仍是 $O(L^2)$ 的乘加，但**访存量**从 $O(L^2)$ 降到 $O(L^2 d^2 / M)$ 量级，其中 M 是 SRAM 大小、d 是 head_dim；直觉上就是「每个 HBM 字节被搬上片后复用了更多次」）。由于注意力在长序列下是访存瓶颈而非算力瓶颈，实际效果是全流程 2–4 倍加速、显存占用从 $O(L^2)$ 降到 $O(L)$——后者让长上下文训练与推理在工程上真正可行。

工程含义。 今天 FlashAttention 已经是所有主流训练与推理框架的默认注意力实现，你不需要手写它，但需要知道两件事：第一，**序列长度 L 的成本模型因它而改变**——算力仍是平方，但显存和访存近似线性，所以「要不要开 FlashAttention」不再是问题，「KV cache 与批大小怎么配」才是

（回到 7.2）；第二，它再次印证本章主线：推理性能的瓶颈往往是 **IO（访存与传输）而不是算**，优化手段都围绕「少搬数据」展开。

7.5 投机解码：小模型起草，大模型并行验收

直觉。 Decode 的死结是串行：第 t 个 token 没出来，第 $t+1$ 个就没法算，大模型一次前向只能换 1 个 token。但换个角度：大模型一次前向其实可以**并行地验证任意多个候选 token**——给它一段序列，它一步就能算出每个位置的下一个 token 分布（这正是 Prefill 在做的事）。于是有了一个漂亮的分工：用一个便宜的小模型快速「起草」一串 token，再让大模型一次前向把它们全部「验收」。这就是**投机解码**（speculative decoding）。

类比。 这是流水线里的「投机执行」，也是你熟悉的「乐观锁」：先假设小模型的草稿是对的，批量提交；大模型校验，对到哪算哪，第一个错的位置用大模型自己的结果修正，后面的作废重来。乐观锁在无冲突时省掉所有协调开销，投机解码在高接受率时省掉大量大模型前向。

机制。 一轮循环：

1. **起草**：小模型（如 1B 以下的草稿模型，或干脆用大模型自己的浅层）自回归连跑 $k=4$ 步，产出 4 个候选 token：t_1, t_2, t_3, t_4；
2. **验收**：把「已有上下文 + 4 个候选」拼成一段，大模型**一次前向**，并行得到 5 个位置的分布（每个位置预测下一个 token）；
3. **逐个比对**：从 t_1 开始，若大模型在该位置也认可这个 token（贪心场景：大模型的 argmax 恰好是它），则接受，继续看下一个；遇到第一个不认可的位置 t_i，用大模型在该位置的输出**修正**它，后面的 $t_{i+1} \dots t_4$ 全部丢弃；
4. 无论接受几个，本轮至少前进 1 个 token（修正的那个），最多前进 $k+1=5$ 个（4 个全接受 + 大模型多送的 1 个）。

关键性质：输出分布与「大模型自己逐个生成」**完全一致**——大模型全程在把关，小模型只影响速度不影响质量。这是它与蒸馏、量化等「有损加速」的本质区别。

手算例子。 设 $k=4$，平均接受 3 个。那么一轮的开销是：小模型 4 次前向（假设每次成本只有大模型的 $1/50$，合计约 0.08 次大模型前向，可忽略）+ 大模型 1 次前向；产出是 $3+1=4$ 个 token（3 个接受 + 1 个修正）。对比基线：大模型 4 次前向产 4 个 token。于是 Decode 的理想加速比接近 4 倍（扣除小模型开销，约 $4/1.08 \approx 3.7$ 倍）。一般地，平均接受 a 个时，一次大模型前向产出 $a+1$ 个 token，加速比约 $a+1$。

接受率决定一切。 草稿越像大模型的输出，接受率越高：小模型与大模型同族（同 tokenizer、同数据分布）、任务越「模式化」（代码补全、格式化输出、复读型摘要接受率高；开放式创作接受率低），收益越大。接受率掉到 1 以下，加速比不到 2 倍，再扣掉草稿开销就未必划算。工程上还有自适应调整 k、用树状草稿（一次验多个分支）等进阶手段，核心账都是这本：接受率 $\times$ 草稿成本。

工程含义。 第一，投机解码优化的是 **TPOT/单请求延迟**，不是吞吐——batch 很大时大模型前向已经吃满算力，验 5 个位置反而多算 5 倍的激活，高负载集群可能关掉它；它最适合低并发、对延迟敏感的在线场景。第二，它不改变 KV cache 的账本（A4 照算），但和连续批处理可以叠加：被丢弃的草稿 token 的 KV 要回滚，这正是分页式 KV 管理（7.2）干起来顺手的事。

7.6 PD 分离：推理集群的「读写分离」

直觉。 到目前为止的优化都在单机之内。拉到集群视角，第 6 章的结论变得刺眼：Prefill 是计算密集（并行处理整个 prompt，算术强度高），Decode 是访存密集（逐 token、读权重读 KV，算力闲置）。两种负载对硬件的诉求完全不同，却被塞进同一批 GPU 里互相干扰——一个长 prompt 的 Prefill 跑进来，正在 Decode 的所有请求 TPOT 集体抖动。

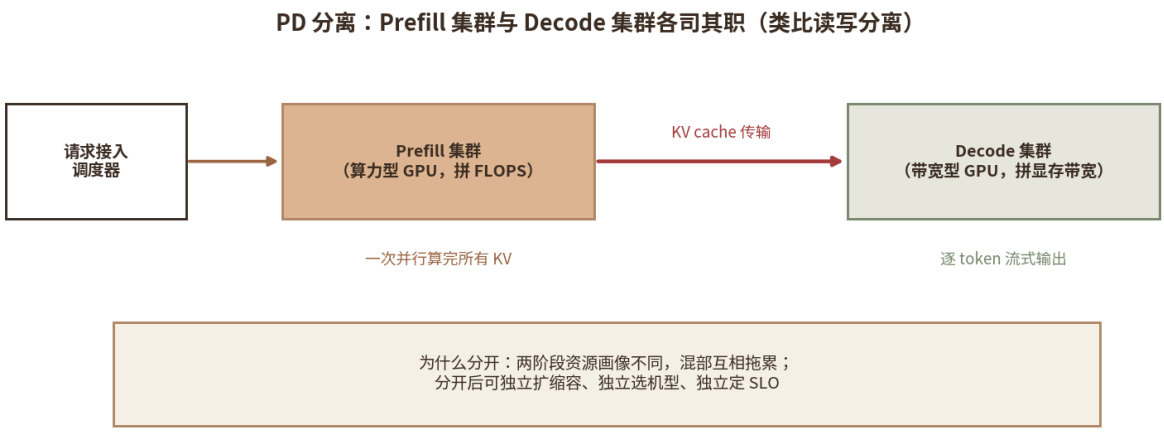
类比。 这就是数据库的**读写分离**。写库（Prefill）处理突发、重计算的首 token；读库（Decode）承载稳定、高频的逐 token 输出；两者之间通过复制（这里是 KV cache 传输）同步状态；读库写库各自独立扩缩容——读多就加读库，写多就加写库。

机制。 PD 分离（Prefill/Decode disaggregation）把推理集群拆成两池：

- **Prefill 池**：接到请求后跑 prompt 的前向，产出首 token 和完整的 KV cache。它要的是算力，可以用更激进的 TP、更大的 chunked batch 来堆 MFU。
- **KV 传输**：Prefill 产出的 KV cache 通过网络送到某个 Decode 实例。按 A4，7B 模型 4096 token 上下文的 KV 约 512 MB，在机间 50 GB/s（A3）的链路上理论传输约 10 ms；若在机内 NVLink 域（900 GB/s）则不到 1 ms。注意这笔开销要计入 TTFT，所以调度器倾向把 KV 传往同机/同机架的 Decode 实例，且可以与逐层 Prefill 流水起来（算完一层传一层）。
- **Decode 池**：接手 KV 后进入连续批处理循环，逐 token 流式输出。它要的是显存容量与带宽，可以配更小的 TP、更大的 batch。

各自独立扩缩容：流量里 prompt 长、输出短（如检索问答）就多配 Prefill；prompt 短、输出长（如代码生成）就多配 Decode。还能异构——Prefill 用算力强的卡，Decode 用显存大的卡，各取所长。

工程含义。 第一，PD 分离让 TTFT 和 TPOT 的 SLO 可以**独立保障**：Prefill 池排队深度决定 TTFT，Decode 池 batch 水位决定 TPOT，互不踩踏；同池部署下这两者永远此消彼长。第二，它引入了你再熟悉不过的分布式问题：KV 传输失败怎么办（重传或回源重算）、Decode 实例挂了怎么办（请求迁移到持有副本的实例，或从 prefix cache 重建）、两池容量怎么配比（按真实流量的 prompt/output 长度分布做容量规划）。第三，PD 分离与 prefix caching 天然合拍：Prefill 池同时是「KV 生产基地」，热点前缀的 KV 可以直接在池内缓存复用，命中就跳过 Prefill 直发 Decode。



7.7 一个推理请求的完整旅程

把本章所有零件装起来，看一个请求从 HTTP 到流式输出的全链路。假设用户向一个 7B 模型服务发了一段 1000 token 的 prompt，要求生成约 200 个 token。

第 1 步，HTTP 接入与鉴权限流。 请求到达 API 网关：校验 API key、查配额与限流（按用户/租户的 QPS 与 token 预算，就是你熟悉的令牌桶）、做请求校验（最大上下文长度、采样参数合法性）。这一步全在 CPU 上，毫秒以内。被限流的请求直接返回 429，不占用任何 GPU 资源。

第 2 步，tokenization。 网关或推理前端的 tokenizer 把文本切成 token 序列，长度 1000。这个数字从此决定一切：它决定 Prefill 的算力开销（约 $2N \times 1000$ FLOPs，A5 口径的每 token $2N$），决定 KV cache 的初始占用（$1000 \times 128\ \text{KB} = 125\ \text{MB}$，A4），也决定计费。

第 3 步，调度器排队。 请求进入调度器的等待队列。调度器扫一眼当前水位：Prefill 池的空闲算力、各 Decode 实例的 KV 块池余量（7.2 的块，而不是连续显存）、以及 prefix cache 是否命中——如果这个请求与某个历史请求共享前缀，命中的块直接复用，Prefill 只需算差分部分。假设本次未命中，请求被派给一台 Prefill 节点，TTFT 计时开始。

第 4 步，Prefill。 Prefill 节点对 1000 个 token 做一次并行前向：FlashAttention 分块在 SRAM 里算注意力，$L \times L$ 矩阵从不落地（7.4）；量化权重边读边反量化（7.3）。前向完毕，产出两样东西：首 token（立刻可以流式推给客户端，TTFT 停表，目标如第 6 章的 500 ms 内）和 125 MB 的 KV cache。

第 5 步，KV 传输。 Prefill 节点把 KV cache 按块发给调度器指定的 Decode 节点。125 MB 在 50 GB/s 机间链路上约 2.5 ms（实际与逐层计算流水重叠，几乎藏在 Prefill 里）。Decode 节点在自己的块池里登记这些物理块，填好 block table。

第 6 步，Decode 循环与流式输出。 请求进入 Decode 节点的连续批处理：每个 iteration，它与几十上百个请求一起跑一步前向，产出 1 个 token（若启用投机解码且草稿质量高，可能一次验收出 3–

4 个，7.5），通过 SSE/WebSocket 流式推给客户端——用户看到的是逐字蹦出的回答。每跑完一步，调度器检查：生成了 EOS 或到 200 token 上限了吗？到了就移除请求、回收 KV 块，空出的容量立刻给队列里的下一个请求（7.1）。显存水位告急时，低优先级请求被抢占，KV 换出到 CPU 内存，稍后恢复（7.2）。

第 7 步，收尾与记账。 连接关闭，计费系统按 token 数记账，监控系统记下这次请求的 TTFT、平均 TPOT、排队时长、KV 命中率——它们汇入第 8 章要讲的推理侧 SLO 大盘。一次请求的旅程结束，而集群里的调度循环，从不停歇。

回看全章：连续批处理解决「调度」，PagedAttention 解决「显存」，量化解决「带宽」，FlashAttention 解决「算子 IO」，投机解码解决「串行」，PD 分离解决「集群负载配比」。每一项你都能在分布式后台的经验里找到对应物——这正是第三册想让你相信的事：AI Infra 不是新大陆，是你的旧地图换了投影。

第 8 章 集群、成本与转型路线图

前七章我们一路向下钻：从一次参数更新的微积分，到千卡并行的通信账本，再到推理引擎里一个 token 的旅程。本章我们拉回到机房的俯瞰视角，回答三个非常"后台工程师"的问题：这套系统跑在什么物理网络上？它一天要花多少钱？以及——你手里攒了八年的分布式功力，怎么平移到这个新战场？

8.1 GPU 集群拓扑：并行策略的物理边界

直觉：并行策略不是凭空设计的，它是网络拓扑的"果"。第 4 章我们反复强调"TP 放机内、PP 跨节点"，本章把这句话背后的硬件讲清楚。

类比：你在后台时代设计过机房网络——机架内服务器走 ToR 交换机，机架间走汇聚层，带宽逐级收敛。GPU 集群的结构几乎一样，只是每一层的带宽数字大了几个数量级，而应用对带宽的饥渴程度也大了几个数量级。

8.1.1 节点内：8 卡 NVLink 全互联域

一台 GPU 服务器通常装 8 张 GPU（附录 A3 口径），8 卡之间通过 NVLink/NVSwitch 组成**全互联域**：任意两张卡之间的点对点带宽约 900 GB/s，且一张卡可以同时对所有其他卡收发。类比后台系统，这就像 8 个服务实例跑在同一台宿主机上、通过共享内存通信——延迟微秒级、带宽几乎"不要钱"。

NVLink 域有两个工程含义。第一，它是**显存池化的边界**：8 张 80 GB 的卡合计 640 GB，通过 TP 切分可以把这 640 GB 当成一块"逻辑大显存"用（第 4 章 TP、第 7 章大模型推理的多卡部署都依赖这一点）。第二，它是**高频小消息通信的保护区**：TP 每层每个矩阵乘都要做一次 AllReduce/AllGather，通信次数以万计，只有 900 GB/s 级的带宽扛得住这种频率。

8.1.2 机间：IB/RoCE 与胖树

跨节点靠 InfiniBand 或 RoCE（RDMA over Converged Ethernet），单网卡 400 Gb/s $=$ 50 GB/s（附录 A3）。注意单位：Gb 是比特、GB 是字节，差 8 倍，报价单上混用这两个单位是新手最容易踩的坑。

RDMA 一句话：RDMA 让网卡直接读写对端机器的显存/内存，全程不经过 CPU 和内核协议栈，类比就是"绕过业务进程、让网卡硬件替你 memcpy"，这是 NCCL 机间通信低延迟的物理基础。

训练节点通常是**多轨网络**：8 张 GPU 各配一张 400 Gb/s 网卡，单节点出口 $8 \times 400 = 3200$ Gb/s，每张卡的机间通信互不抢道——类比就是给每个微服务实例单独拉一根专线，而不是 8 个实

例挤一个网口。采购时务必区分这一点：标称同样的 GPU 机型，单网卡版与八网卡版的机间带宽差 8 倍，跑起 DP 的 AllReduce 是天壤之别。

多机互联的标准拓扑是**胖树（Fat-Tree）**：接入层交换机下联 GPU 节点、上联汇聚层，逐层收敛。关键指标是**收敛比（oversubscription ratio）**：

- **无阻塞 1:1**：接入层的上行总带宽 = 下行总带宽，任意两台机器之间都能跑满 400 Gb/s。训练集群要求这个规格，因为 AllReduce 是全局流量，任何一处收敛都会成为全体木桶的短板。
- **超售** $2:1$ / $4:1$：上行带宽只有下行的一半/四分之一。通用云数据中心常见，省钱；但对训练意味着跨交换机的 AllReduce 带宽腰斩，MFU 直接受损。推理集群的 PD 分离架构（第 7 章）对跨机流量同样敏感，但同层 Decode 节点之间流量小，容忍度略高。

8.1.3 手算：拓扑如何"决定"并行映射

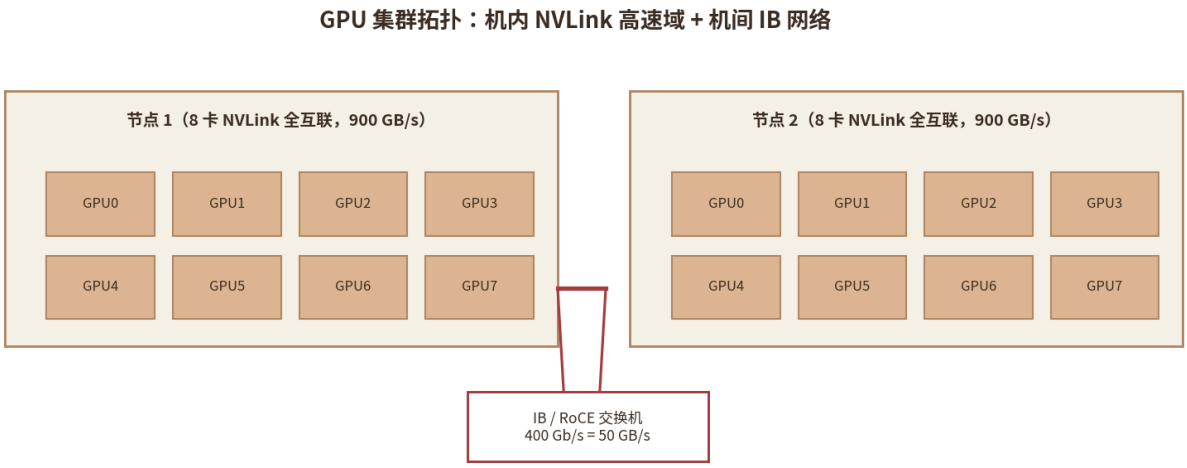
把两档带宽摆在一起：

$$\frac{\text{NVLink}}{\text{IB}} = \frac{900\ \text{GB/s}}{50\ \text{GB/s}} = 18$$

18 倍的带宽差，就是第 4 章那条经验法则的全部来源：

- **TP（张量并行）**：每层至少 2 次集合通信，一个 32 层模型每步上百次，消息小而频繁 $\rightarrow$ 只能放在 NVLink 域内（8 卡以内）。
- **PP（流水线并行）**：只在相邻流水级之间传激活值，一个 micro-batch 每级只传一次，通信频率低两个数量级 $\rightarrow$ 可以跨节点走 IB。
- **DP（数据并行）**：每个 iteration 末尾一次梯度 AllReduce，消息大（7B 模型 bf16 梯度 14 GB）但频率最低，还能用 bucket 化通信与反向计算重叠（第 5 章）$\rightarrow$ 放在最外层、跨最多节点。

工程含义：拿到一个集群的第一件事不是装驱动，而是画出带宽地图——哪些机器之间是 900 GB/s，哪些是 50 GB/s，哪些跨了超售的汇聚层。并行策略（TP=8 机内、PP 跨 2 机、DP 铺满）本质上就是把通信频率从高到低的三种并行，对号入座地贴到带宽从高到低的链路上。拓扑变了（比如换成 NVLink 域更大的机型），最优并行映射就要重新算。



8.2 存储与数据管道：喂饱 GPU，接住 Checkpoint

GPU 集群的存储系统要同时满足两种脾气截然相反的负载：**读**是持续的、分散的高吞吐流，**写**是罕见的、集中的 TB 级风暴。

8.2.1 训练数据读取：集群级 TB/s 聚合

直觉：GPU 算得快，DataLoader 就得喂得更快，否则 20 元/卡时的算力在等 IO。

类比：这就是你熟悉的"算存分离"——数据在对象存储里，计算节点本地缓存热点，和后台的"对象存储 + CDN + 本地磁盘缓存"三级结构一模一样，只是吞吐目标从 Gb/s 变成了 TB/s。

手算：设每张卡消费预处理后的 token 序列约 300 MB/s（packed 序列、bf16、含一定的预取冗余，数量级口径）。1024 卡集群的聚合读带宽：

$$300\ \text{MB/s} \times 1024 \approx 300\ \text{GB/s} \approx 0.3\ \text{TB/s}$$

如果是 4096 卡的超大集群，需求直接到 1.2 TB/s 量级——这已经超过单套并行文件系统的经济甜点，所以工业界的标准解法是分层：**对象存储存全量语料（便宜、无限扩容）→ 并行文件系统或本地 NVMe 做热缓存 → DataLoader 预取流水线**。第 3 章讲的 packing、shuffle 分片、多 worker 预取，都是为了让这条链路在对象存储高延迟的脾气下仍能喂饱 GPU。

两个后台老兵容易低估的坑：其一，**对象存储怕小文件**——语料必须预先打成几十上百 MB 的大分片（shard），否则海量小文件的元数据开销会把吞吐打到骨折，这和 Kafka 怕小消息是同一个道理；其二，**shuffle 与吞吐的矛盾**——训练要求每个 epoch 全局打乱，而流式读取最讨厌随机 IO，工程上用"分片级打乱 + 分片内缓冲池打乱"的两级近似来折中。

8.2.2 Checkpoint 写入风暴

直觉：checkpoint 是训练系统的"定期存款"——平时不碰，一存就是全部身家。1024 卡在同一时刻一起存，就是一场写入风暴。

手算：7B 模型的 bf16 权重 14 GB（附录 A2），训练侧完整状态（fp32 主权重 + 梯度 + AdamW 动量）约 112 GB，合计 126 GB 的全局状态经 ZeRO/FSDP 切分后每卡只承担一小片。但实际 checkpoint 还会按 DP 组冗余保存权重副本、附带激活与流水线快照，为容错预留的**每卡落盘量约 14 GB** 是常见的规划口径。于是 1024 卡同一时刻一起写：

$$1024 \text{ 卡} \times 14 \text{ GB} \approx 14 \text{ TB}$$

如果要求 10 分钟内写完，瞬时聚合写带宽需求是：

$$\frac{14 \text{ TB}}{600 \text{ s}} \approx 23 \text{ GB/s}$$

23 GB/s 听起来不大，但这 14 TB 是**同步爆发**的——所有卡在同一秒开始写，存储集群的元数据服务（目录创建、分片分配）和少数热存储节点会先被冲垮，这就像 1024 个实例同时向同一套 MySQL 写大 blob。

分层方案（第 5 章 checkpoint 小节的基础设施面）：

1. **第一跳：本地 NVMe 落盘**。每卡先写本机 SSD，单盘 5 GB/s 以上，14 GB 约 3 秒落盘，训练随即恢复——把"训练停顿时间"从分钟级压到秒级。
2. **第二跳：异步上传对象存储**。后台线程慢慢把本地分片搬运到对象存储，合并成全局 checkpoint。训练进程完全不感知。
3. **保留策略**：对象存储上滚动保留最近 N 份全量 + 每日一份归档，兼顾回滚能力（第 5 章 loss spike playbook 要用）与存储成本。

分层设计在**恢复路径**上同样划算。手算恢复时间：14 TB 若从对象存储以聚合 10 GB/s 拉回，需 $14 \times 10^3 / 10 = 1400$ 秒 ≈ 23 分钟；若本地 NVMe 副本还在（机器没坏、只是进程崩了——这是故障的大多数），单机读回 14 GB 只要约 3 秒。**先查本地副本、再查并行文件系统、最后才回源对象存储**，这条三级回退链把平均恢复时间从半小时压到分钟级，和第 5 章的弹性训练配合，才能把 1024 卡集群"每天都有硬件故障"的现实消化掉。

对照一下你熟悉的概念：

分布式后台	AI Infra 对应物
binlog/定期快照	checkpoint
本地落盘 + 异步同步到远端	分层 checkpoint
对象存储（S3）	语料与 checkpoint 的最终归宿

CDN/本地缓存	DataLoader 的分层数据缓存

8.3 成本核算：一笔账算清训练与推理

AI Infra 工程师的每个技术决策，最后都会落到一张成本表上。本节做两道手算：训练一个 7B 多少钱，推理一百万 token 多少钱。你熟悉的容量规划技能在这里完全复用，只是资源从"CPU 核数"换成了"卡时"。

8.3.1 训练成本：A5 续算

直觉：训练成本 = 卡时消耗 × 卡时单价。卡时消耗由总 FLOPs 和 MFU 决定（第 2、5 章已算），本节只做"换算成钱"的最后一步。

手算：附录 A5 已算出 7B 模型、2T tokens、MFU 50% 时，1024 卡需要约 1.9 天。卡时总量：

$$1024 \text{ 卡} \times 1.9 \text{ 天} \times 24 \text{ 小时} \approx 4.67 \times 10^4 \text{ 卡时} \approx 4.7 \text{ 万卡时}$$

设 H100 级 GPU 时租 20 元/卡时（**示例假设**：真实价格随市场波动剧烈，裸金属长租、云上按需、竞价实例可能相差数倍，读者用时请自行替换）：

$$4.7 \times 10^4 \text{ 卡时} \times 20 \text{ 元/卡时} \approx 94 \text{ 万元}$$

一个 7B 预训练，约 94 万元。三个工程含义：

1. **MFU 就是钱**。MFU 从 50% 掉到 40%，同样 8.4×10^{22} FLOPs 要跑 $1.9 \times 50/40 \approx 2.4$ 天，成本涨到约 117 万——第 5 章那些通信重叠、梯度检查点取舍，每一条都对应几十万的差额。
2. **这只是"一遍成功"的价格**。loss spike 回滚、硬件故障重启、超参试错，实际预算通常按理论值的 $1.5 \sim 2$ 倍准备——8.2 节的分层 checkpoint 和 8.4 节的监控，买的都是这份保险。
3. **卡时账单里不含**数据工程、电力、网络、存储与人力，做预算时别忘了这些"隐藏列"。

顺手做一次 scaling 估算：按 $6ND$（第 2 章），模型从 7B 涨到 70B，同样 2T tokens 的 FLOPs 线性放大 10 倍，卡时约 47 万、成本约 940 万——**预训练成本对参数量近似线性**，这就是"大模型军备竞赛"在财务表上的样子。反过来，只把数据量翻倍而模型不变，成本也只翻倍，但效果增益遵循 scaling law 的对数曲线（第二册）——这条不对称是所有"该不该继续堆数据"决策的起点。

8.3.2 推理成本：每百万 token 的定价模型

直觉：推理是持续服务，成本按"单位产出"算——就像后台系统算"每次请求的成本"。产出单位是 token。

类比：这就是一次容量规划练习：单实例 QPS 决定需要多少实例、实例单价决定总成本，总成本除以请求量就是单价。

手算：设单卡 Decode 吞吐 1000 token/s（7B、中等并发下的示例口径，第 6 章说过 Decode 是访存密集型），单卡一小时产出：

$$1000\ \text{token/s} \times 3600\ \text{s} = 3.6 \times 10^6\ \text{token}$$

每百万 token 的成本公式：

$$\text{每百万 token 成本} = \frac{\text{卡时单价}}{\text{吞吐} \times 3600} \times 10^6$$

代入 20 元/卡时：

$$\frac{20}{1000 \times 3600} \times 10^6 = \frac{20}{3.6} \approx 5.6\ \text{元/百万 token}$$

注意 10^6 恰好和 1000×3600 里的数量级相消，这类心算估算在评审会上非常好用：**成本（元/百万 token）≈ 卡时单价 ÷ 3.6 ÷ 吞吐（千 token/s）**。

吞吐对成本的杠杆是线性的，做一张敏感性小表（同为 20 元/卡时）：

单卡吞吐（token/s）	每百万 token 成本
200（静态 batch）	≈ 28 元
1000（continuous batching）	≈ 5.6 元
2000（叠加量化/投机解码）	≈ 2.8 元

工程含义：第 7 章的每一项技术都可以翻译进这个公式——连续批处理把吞吐从 200 提到 1000 token/s，成本直接除以 5；量化让单卡能放下更大的 batch，投机解码提高有效吞吐，PD 分离让两类节点各自扩容不浪费。推理工程师的 OKR，本质就是让上面这个分式的分母变大。另外别忘了 TTFT/TPOT 的 SLO 约束：吞吐可以靠堆并发拉满，但 P99 延迟是天花板，两者围出的区域才是真实可行的成本区间。

8.4 监控与 SLO：训练看健康，推理看体验

类比：后台监控的金字塔你闭着眼都能背——RED（Rate/Error/Duration）看服务，USE（Utilization/Saturation/Errors）看资源。AI Infra 的监控完全可以套这个骨架，只是指标换了一批主角，而且训练侧与推理侧是**两套性格不同的监控体系**。

8.4.1 训练侧：一场数周长跑的生命体征

训练任务跑几天到几周，监控的目标是**尽早发现"在烧钱但没在学"的状态**。核心指标：

指标	含义	异常模式	对应处置

loss 曲线	模型是否在收敛	突刺（spike）、发散	回滚 checkpoint、跳 batch、降 lr（第 5 章 playbook）
梯度范数	更新力度是否稳定	爆增或归零	梯度裁剪失效排查、数据脏样本定位
MFU	算力利用率	下降 $>10\%$	查通信热点、慢节点、数据饥饿
NCCL 超时/错误	通信健康度	成片超时	定位故障网卡/节点，剔除并重调度
checkpoint 成功率	容错底座是否可靠	一次失败	立即修复——下次故障就没有退路了
数据管道吞吐	喂数是否跟得上	DataLoader 等待占比上升	加 worker、查缓存命中率

训练监控还有一个后台世界没有的角色：**慢节点（straggler）检测**。AllReduce 是同步操作，全体等最慢的那张卡——一张网卡降速的 GPU 能把 1024 卡的 MFU 拖低 10%。工程上监控每个 rank 的通信等待时间，其**分布的尾部**（而非均值）是慢节点的指纹；定位到后把它剔除出调度池，和第 5 章的弹性训练联动。这和你熟悉的"摘除抖动下游实例"是同一个动作，只是同步屏障把这个问题的代价放大到了全集群。

8.4.2 推理侧：用户看得见的体验指标

推理是在线服务，监控目标对齐第 6 章定义的 SLO：

指标	后台对应物	关注点
TTFT P99	首字节延迟 TTFB	用户感知的第一下卡顿
TPOT P99	流式响应的帧间隔	生成过程是否"一卡一卡"
排队时长	请求在队列中的等待	容量不足的先兆，先于延迟恶化
KV cache 显存水位	连接池/内存水位	接近上限时会触发抢占或拒绝，导致延迟陡增
吞吐（token/s）与 goodput	QPS 与有效 QPS	goodput 只计满足 SLO 的请求，是唯一诚实的容量指标

8.4.3 告警分级：按"烧钱的不可逆程度"分级

后台的分级看业务影响面，AI Infra 的分级看**损失是否可恢复**：

- **P0（立即响应）**：训练中断（进程退出、NCCL 成片超时、loss 发散）——每一分钟都是 1024 卡白烧，$94\text{ 万}/1.9\text{ 天}\approx 2\text{ 万元}/\text{小时}$ 的空转成本；推理侧对应核心 SLO 全面击穿。
- **P1（小时级响应）**：MFU 下降超阈值、checkpoint 失败、loss 出现 spike 但已自愈——任务还在跑，但性价比或安全性已受损。

- **P2（工作时段处理）**：吞吐缓慢劣化、显存水位趋势性上涨、数据管道命中率下降——趋势性问题，纳入容量规划。

一个易忽略的点：训练任务的告警**值班人必须有处置权限**（重启、剔除节点、回滚 checkpoint）。只告警不给 playbook 和权限，等于让值班盯着 2 万元/小时的火苗报警。

8.5 转型路线图：把八年功力平移过来

8.5.1 能力迁移表

回到第 1 章的那句话：换个战场，不换基本功。通读全书后，我们可以把这张对照表填满了——左边是你已经会的东西，右边是它在 AI Infra 里的样子：

分布式后台技能	AI Infra 对应物	所在章节
RPC / 负载均衡	集合通信（AllReduce/AllGather）；推理服务网关与调度器	第 1、4、7 章
分片与复制	DP/TP/PP/ZeRO 四类并行切分，副本一致性变成梯度同步	第 4 章
缓存（Redis/多级缓存）	KV cache、前缀缓存、数据分层缓存	第 6、7、8 章
容量规划	MFU 核算、卡时预算、每百万 token 成本模型	第 5、8 章
SLO 与监控	TTFT/TPOT/goodput 体系、训练生命体征监控	第 6、8 章
混沌工程 / 容灾演练	容错与弹性训练、checkpoint 恢复演练、慢节点剔除	第 5、8 章
消息队列 / 流处理	DataLoader 预取流水线、数据 ETL 管道	第 3 章
操作系统内存管理	PagedAttention 的 KV cache 分页	第 7 章

你缺的不是"分布式系统的直觉"，而是三样新东西：**概率与微积分的语感**（第二册的系统数学）、**GPU 的脾气**（显存、带宽、SIMT），以及**一套新的度量单位**（FLOPs、token、卡时）。前两样靠读书和动手，最后一样靠做几遍本章这样的手算。

8.5.2 90 天计划

第 1–2 周：原理打底。通读本册与第二册。验收标准：能独立手算 A1 的反向传播、A2 的显存账、A5 的算力账——也就是把每章的公式在纸上推一遍，而不是"看懂了"。这一步偷懒，后面所有源码都会读成天书。

第 3–6 周：动手跑起来。两件事：第一，用 HuggingFace + 一张消费级显卡微调一个 1B 以下的小模型，完整走一遍"数据 → 训练 → loss 曲线 → checkpoint → 恢复"的闭环，亲手制造一次 loss spike 并按第 5 章的 playbook 处置；第二，用 vLLM 把同一个模型部署成服务，压测出它的 TTFT/TPOT/吞吐曲线，观察 KV cache 水位上升时延迟如何劣化。验收标准：你能向同事解释清楚每一个曲线的拐点。

第 7–10 周：读源码，找手感。带着问题读，不求全：vLLM 侧读调度器（continuous batching 的主循环）、块管理器（PagedAttention 的页表）；Megatron-LM 侧读 TP 的列/行并行实现和 PP 的 1F1B 调度。每个模块读完用一句话向自己复述："它解决什么问题，代价是什么"。验收标准：给一个线上症状（如 TTFT 抖动），你能说出三个可疑的代码位置。

第 11–13 周：参与真实项目。选一个能上线的小事：给训练任务补齐监控指标与告警分级（8.4 节清单）、给推理服务做一次成本压测并产出优化报告（8.3 节模型）、或者修一个真实的 bug。验收标准：你的改动被合并，并且你在复盘会上能讲清它的性能与成本影响。

90 天后，你不会变成算法研究员，但你会成为团队里那个"既懂系统又懂大模型"的人——这正是 AI Infra 团队最稀缺的画像。

最后提醒三个转型期常见误区：**一**，试图把数学"补完再动手"——高中数学加本册的手算已足够起步，在跑真实任务中补概率与线代效率更高；**二**，只看框架文档不看论文与源码——vLLM、Megatron 的文档滞后于代码，第 7–10 周的源码阅读能力才是真正的护城河；**三**，把后台经验当成要"忘掉重来"的包袱——本章的对照表已经说明，你过往八年积累的容量规划、SLO、容灾直觉，恰恰是多数 ML 背景工程师最缺的那一半。

8.5.3 全书收束

从第 1 章的两个世界对照开始，我们走完了完整的一圈：第 2 章把一个参数的更新拆成加减乘除，第 3 章把一个 token 从网页追到显存，第 4 章用分片与通信把 70 亿参数铺进集群，第 5 章用混合精度、容错和 MFU 守住训练的稳定性与钱包，第 6、7 章把视角切到推理侧，用 KV cache、连续批处理和 PD 分离回答了延迟与吞吐的双重约束，本章则把这一切落到物理拓扑、成本表格和你自己的 90 天计划上。原理让我们知道系统为什么这样设计，工程让我们在真实约束下把它跑稳、跑省——从一次反向传播到一个千卡集群，从一个 token 到一份 SLO，这条从原理到工程的闭环，就是你转型路上最可靠的地图。

